

Report on

**Intelligent NPCs in Games using
Reinforcement Learning**

Submitted to:

University Institute of Computing

Submitted by:

Manish

(24MCI10140)

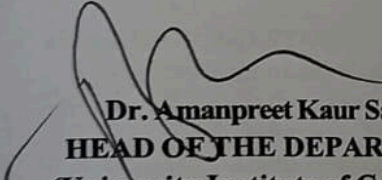


Chandigarh University, Mohali

BONAFIDE CERTIFICATE

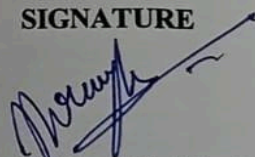
Certified that this project report "**Intelligent NPCs in Games using Reinforcement Learning**" is the Bonafide work Of "**Manish**" who carried out the project work under my/our supervision.

SIGNATURE



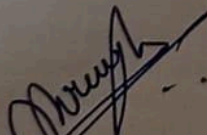
Dr. Amanpreet Kaur Sandhu
HEAD OF THE DEPARTMENT
(University Institute of Computing)

SIGNATURE

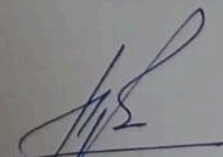


Mr. Dhruv Shah Samir
SUPERVISOR
(Assistant professor)
(University Institute of Computing)

Submitted for the project viva-voce examination held on



INTERNAL EXAMINER



EXTERNAL EXAMINER

BONAFIDE CERTIFICATE

Certified that this project report “**Intelligent NPCs in Games using Reinforcement Learning**” is the Bonafide work Of “**Manish**” who carried out the project work under my/our supervision.

SIGNATURE

Dr. Krishan Tuli
HEAD OF THE DEPARTMENT
(University Institute of Computing)

SIGNATURE

Mr. Dhruv Shah
SUPERVISOR
(Assistant professor)
(University Institute of
Computing)

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER 1.....	1
INTRODUCTION.....	1
1.1 Need Identification.....	2
1.1.1 Limitations of Finite-State Machine NPCs.....	2
1.1.2 Limitations of Behaviour Trees.....	3
1.1.3 The Reinforcement Learning Alternative.....	3
1.1.4 Identified Needs.....	4
1.2 Identification of Problem.....	5
1.2.1 Reward Shaping for Multi-Objective Combat Behaviour.....	5
1.2.2 Curriculum Transition Stability.....	5
1.2.3 Self-Play Drift in Deathmatch Settings.....	5
1.2.4 Dual-Domain Architecture.....	6
1.2.5 Performance Constraints.....	6
1.2.6 Integration Complexity.....	6
1.3 Identification of Tasks.....	6
1.3.1 AI and Learning Task Group.....	7
1.3.2 Environment and Gameplay Task Group.....	7
1.3.3 Systems Integration Task Group.....	8
1.3.4 User Interface and Tooling Task Group.....	8
1.3.5 Validation and Analysis Task Group.....	9
1.4 Project Timeline.....	9
1.5 Organisation of the Report.....	11
CHAPTER 2.....	12
BACKGROUND STUDY.....	12
2.1 Timeline of the Reported Problem.....	12
2.1.1 Early Game AI: Rule-Based NPCs (Pre-2000s).....	12
2.1.2 Behaviour Trees and Utility Systems (2000s-2010s).....	13
2.1.3 Deep Reinforcement Learning Enters Games (2013-2018).....	14
2.1.4 Multi-Agent and Self-Play Expansion (2018-2022).....	15
2.1.5 Curriculum Learning for Game Agents.....	16
2.1.6 Practical Integration and Hybrid Approaches (2022-Present).....	17
2.2 Proposed Solution.....	18
2.2.1 Learned Combat Agent.....	18
2.2.2 Scripted Baseline Opponent.....	18
2.2.3 Learned Racing Agent.....	18
2.2.4 Manager Orchestration.....	19
2.2.5 UI and Tooling Layer.....	19
2.3 Bibliometric Analysis.....	20
2.3.1 Methodology.....	20
2.3.2 Key Findings: Publication Trends.....	20
2.3.3 Most Significant Authors and Papers.....	22
2.3.4 Thematic Clusters.....	23
2.3.5 Research Gap Analysis.....	23
2.4 Review Summary.....	24

2.5 Problem Definition.....	25
2.6 Goals and Objectives.....	25
2.6.1 Primary Objectives.....	25
2.6.2 Secondary Objectives.....	26
2.7 Technology Selection Rationale.....	26
CHAPTER 3.....	29
DESIGN AND PROCESS.....	29
3.1 System Architecture Overview.....	29
3.1.1 Agent Layer.....	29
3.1.2 Environment Layer.....	32
3.1.3 Combat Systems Layer.....	33
3.1.4 Manager Orchestration Layer.....	34
3.1.5 UI and Operations Layer.....	35
3.2 Evaluation and Selection of Specifications.....	36
3.2.1 Software and Hardware Requirements.....	36
3.2.2 Unity Package Dependencies.....	37
3.2.3 Training Configurations.....	37
3.2.4 Codebase Profile.....	38
3.2.5 CombatAgent Observation Space.....	38
3.2.6 CombatAgent Action Space.....	39
3.3 Design Constraints.....	40
3.4 Design Flow.....	41
3.4.1 Combat Curriculum Flow.....	41
3.4.2 Deathmatch Self-Play Flow.....	42
3.4.3 Racing Agent Flow.....	43
3.5 Data Flow Design.....	44
3.5.1 Combat Observation-Action-Reward Cycle.....	45
3.5.2 Racing Observation-Action-Reward Cycle.....	46
3.5.3 Reward Function Design.....	46
3.6 User Interface Design.....	48
3.6.1 Combat HUD Design.....	48
3.6.2 Deathmatch HUD Design.....	49
3.6.3 Racing HUD Design.....	50
3.6.4 Training GUI Design.....	51
3.7 Implementation Plan and Methodology.....	52
CHAPTER 4.....	54
RESULT ANALYSIS AND VALIDATION.....	54
4.1 Implementation of Solution.....	54
4.1.1 Combat Agent Implementation Details.....	54
4.1.2 Scripted Baseline and Hybrid Behaviour.....	55
4.1.3 Deathmatch Manager Implementation.....	56
4.1.4 Racing Agent Implementation.....	57
4.1.5 Training GUI Implementation.....	59
4.2 Testing and Validation.....	60
4.2.1 Run-Level Training Evidence.....	60
4.2.2 Training Runtime Performance.....	60
4.2.3 Detailed Checkpoint Analysis: phase1_v3.....	61
4.2.4 Self-Play Analysis: Deathmatch Runs.....	62
4.2.5 Long-Run Analysis: combat_phase1.....	63

4.2.6 Gameplay Validation.....	64
4.2.7 Integration Validation Matrix.....	65
4.3 Performance Analysis.....	66
4.3.1 Training Throughput.....	66
4.3.2 Runtime Inference Performance.....	66
4.3.3 Comparison with Related Work.....	67
4.4 Ethical and Gameplay Fairness Notes.....	68
4.5 Usability Evaluation.....	68
4.6 Current Limitations.....	70
CHAPTER 5.....	72
CONCLUSION AND FUTURE WORK.....	72
5.1 Conclusion.....	72
5.2 Future Work.....	74
5.2.1 Racing Metrics Completion.....	74
5.2.2 Multi-Objective Reward Auto-Tuning.....	74
5.2.3 Self-Play Population Management.....	75
5.2.4 Multi-Scene Domain Randomisation.....	75
5.2.5 Inference Mode Benchmarking.....	75
5.2.6 Behaviour Cloning Bootstrap.....	76
5.2.7 Tactical Team Communication.....	76
5.2.8 Dynamic Difficulty Adaptation.....	76
5.2.9 Explainability Tools.....	76
5.2.10 Automated Regression Testing.....	77
5.2.11 Model Versioning.....	77
5.2.12 Production Packaging.....	77
REFERENCES.....	78

ABSTRACT

This report documents the design, implementation, and evaluation of intelligent non-player characters (NPCs) trained through reinforcement learning for two gameplay domains: tactical combat and autonomous racing. The system is built in Unity 6 (Editor version 6000.3.8f1) using the Unity ML-Agents toolkit (version 2.0.2) with Proximal Policy Optimisation (PPO) as the primary training algorithm.

The combat intelligence system trains a `CombatAgent` through a seven-stage curriculum progressing from basic navigation to full team deathmatch with competitive self-play. The observation space comprises 37 scalar values supplemented by ray perception sensors, and the action space includes four continuous actions for movement and aiming alongside two discrete actions for shooting and reloading. A `ScriptedBot` finite-state machine provides intermediate-difficulty opponents during curriculum stages 4 and 5, with self-play replacing scripted opponents in stage 6.

The racing intelligence system trains a `CarDrivingAgent` with a 25-dimensional observation space combining car state variables, raycast-based track perception, and checkpoint direction vectors. The agent learns throttle, steering, and drift control through a four-stage curriculum progressing from straight-line driving to competitive racing.

Manager orchestration systems (`GameManager`, `DeathmatchManager`, `RaceManager`) coordinate episode boundaries, team assignment, respawn logic, and score tracking. Three HUD systems (`CombatHUD`, `DeathmatchHUD`, `RacingHUD`) provide real-time gameplay feedback. A Python-based `TrainingGUI` supports training workflow management with configuration selection, run-id management, and real-time log monitoring.

Six training runs were completed and analysed. The `phase1_v3` run achieved the strongest reward profile (89.96 at step 5,042,737) with successful curriculum progression through all five combat lessons. Self-play deathmatch runs produced competitive behaviour with ELO tracking reaching 1035.29. The codebase comprises 43 C# scripts across nine module directories, totalling approximately 8,743 lines of source code.

LIST OF FIGURES

- Fig 1: Project workspace and core module layout
- Fig 2: Module dependency graph showing relationships between AI, Combat, Game, and U...
- Fig 3: Gantt chart of project phases showing parallel tracks and dependencies
- Fig 4: Scene hierarchy showing agents, managers, and gameplay entities
- Fig 5: Publication trends in RL game NPC research (2010-2025) bar chart
- Fig 6: Citation network showing relationships between key papers
- Fig 7: CombatAgent inspector showing Behavior Parameters, sensors, and serialised fi...
- Fig 8: ScriptedBot FSM state diagram showing transitions between Patrol, Chase, Atta...
- Fig 9: CurriculumManager inspector and runtime lesson value during training
- Fig 10: Deathmatch self-play data flow showing agent-manager-opponent interactions
- Fig 11: Car scene overview with track, checkpoints, and AI-controlled vehicle
- Fig 12: Combat observation-action-reward data flow diagram
- Fig 13: Racing observation-action-reward data flow diagram
- Fig 14: CombatHUD element layout diagram showing health, ammo, reticle, and score pos...
- Fig 15: DeathmatchHUD with team score bar, timer, and active kill feed
- Fig 16: RacingHUD with speedometer, lap counter, and position indicator
- Fig 17: TrainingGUI main window showing config panel, command preview, and log monitor
- Fig 18: Team deathmatch runtime view showing both teams in active combat with kill fe...
- Fig 19: CarDrivingAgent raycast fan visualisation showing 17 rays detecting track bou...
- Fig 20: Training Manager GUI showing config selection, run-id, and live log monitor
- Fig 21: phase1_v3 reward curve showing curriculum phase transitions and steady improv...
- Fig 22: Self-play ELO trajectory showing non-monotonic improvement with opponent pool...
- Fig 23: Combat HUD showing health bar, ammo count, and hit marker feedback during gam...
- Fig 24: Racing HUD displaying speed, lap counter, and position during active ra

CHAPTER 1

INTRODUCTION

Games have been training grounds for artificial intelligence since the 1950s, when Arthur Samuel wrote a checkers programme that learned from its own mistakes. But there is a wide gap between board game AI and the kind of intelligence that a modern 3D game demands from its non-player characters. An NPC in a first-person combat game needs to move through irregular terrain, track opponents across occluded sightlines, decide when to shoot and when to take cover, and do all of this at 30 frames per second without looking obviously stupid. A racing NPC needs to brake before corners, find efficient lines through chicanes, and recover from collisions without freezing in place. These are not problems that a lookup table or a hand-authored state machine handles gracefully.

Traditional approaches to game NPC behaviour rely on finite-state machines (FSMs), behaviour trees, and utility-based AI systems. These methods have served the game industry well for decades, producing acceptable NPC behaviour in most commercially shipped titles. However, they share a fundamental limitation: every decision the NPC can make must be explicitly anticipated, authored, and tested by a human designer. As game environments grow in complexity, as player populations develop increasingly sophisticated strategies, and as the demand for emergent, adaptive gameplay increases, the authoring cost of hand-crafted NPC logic scales poorly.

Reinforcement learning (RL) provides an alternative paradigm. Instead of specifying what the NPC should do in every situation, the developer defines what the NPC can observe, what actions it can take, and what outcomes are rewarded or penalised. The policy, the mapping from observations to actions, emerges from simulated experience rather than manual specification. Deep reinforcement learning, combining RL with neural network function approximation, has demonstrated superhuman performance in domains ranging from Atari games to Go to StarCraft II.

This report documents the development of a system called Intelligent NPCs in Game using AI/ML, built in Unity 6 (Editor version 6000.3.8f1) with the Unity ML-Agents toolkit (version 2.0.2). The project trains NPC policies through Proximal Policy Optimisation (PPO) with optional curiosity-driven exploration and competitive self-play.

It covers two gameplay domains: combat intelligence in arena and team deathmatch scenarios, and racing intelligence for track navigation and competitive driving. The codebase contains 43 C# scripts across nine module directories (AI, Car, Combat, Arena, Camera, Game, Player, Editor, UI), totalling approximately 8,743 lines of source code. Three primary scenes (SampleScene, DeathmatchScene, car) and four YAML training configurations support a complete workflow from policy training through in-game deployment. Training outputs are exported as ONNX model files for runtime inference.

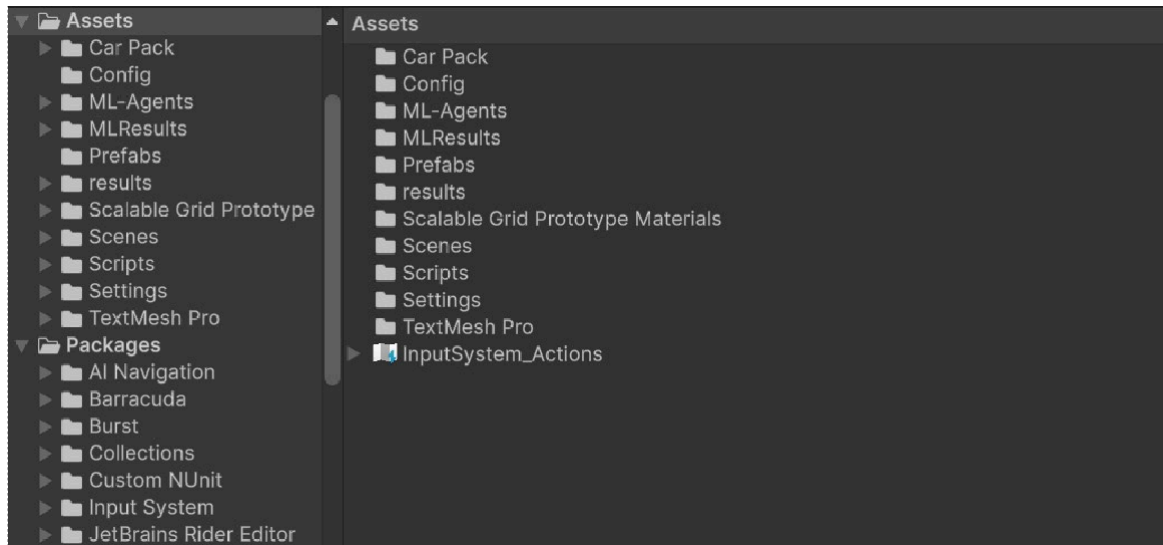


Figure 1.1: Project workspace and core module layout

1.1 Need Identification

The need for this project arises from a convergence of three factors: the growing demand for adaptive NPC behaviour in modern games, the limitations of existing scripted approaches at scale, and the maturation of RL toolkits that make learned NPC policies practically achievable within a game engine.

1.1.1 Limitations of Finite-State Machine NPCs

Most commercial games still ship NPCs that run on finite-state machines. An FSM-based bot follows the same patrol route every round, attacks the nearest target regardless of tactical context, and never adapts to what the player is actually doing. Players figure out the patterns within a few hours and the bot stops being interesting. An FSM for a combat NPC exists in one of a handful of states, typically patrol, chase, attack, and flee. Transitions between states are triggered by simple conditions such as whether the player is visible, whether health is below a threshold, or whether a timer has expired.

The ScriptedBot implementation in this project (475 lines, five states: Patrol, Chase, Attack, Strafe, Flee) illustrates both the utility and the ceiling of this approach. Even with randomised personality parameters (reaction time between 0.05 and 0.4 seconds, aim accuracy between 0.7 and 1.0, strafe probability at 60% base, dodge probability at 30% base), the scripted bot produces behaviour that a player can predict after a few encounters. It cannot learn new strategies, adapt to opponent tactics, or develop emergent coordination with teammates.

FSMs are fast to evaluate at runtime and transparent to designers, which explains their persistence. But they do not scale. A combat NPC with 10 states and 5 possible transition conditions per state already has 50 edges to maintain. Adding a new weapon type or map layout means editing the state machine by hand, testing every branch, and hoping nothing breaks downstream. The authoring cost grows combinatorially with the number of game mechanics.

1.1.2 Limitations of Behaviour Trees

Behaviour trees, popularised by Halo 2 and standardised in engines like Unreal, replaced the flat state graph of FSMs with a hierarchical tree of tasks. A root node selects among child strategies (attack, defend, patrol), each of which decomposes into smaller actions (move to cover, peek, fire). This modular structure makes NPC logic easier to read and extend than equivalent FSMs.

However, behaviour trees still require a designer to manually enumerate every condition and response. A behaviour tree for a combat NPC with ten possible actions in three different tactical contexts can easily grow to hundreds of nodes. The complexity is merely better organised, not reduced. Furthermore, behaviour trees produce deterministic responses to identical situations, making them predictable to observant players. While decorators and randomisation nodes can add variance, the fundamental structure remains reactive rather than adaptive.

1.1.3 The Reinforcement Learning Alternative

Reinforcement learning offers a fundamentally different approach. Instead of writing rules, you define what the agent can observe (its observation space), what actions it can take (its action space), and what reward signal it receives for different outcomes.

The policy, the function that maps observations to actions, emerges from experience rather than specification.

An RL-trained combat NPC can learn to strafe behind cover, lead its shots against moving targets, and retreat when health is low, all without anyone programming those behaviours explicitly. The tradeoff is that RL policies are expensive to train, sensitive to reward shaping, and can collapse or diverge without careful curriculum design. The training process requires significant computational resources and careful monitoring, but the resulting policies can exhibit emergent behaviours that would be extremely difficult to author manually.

1.1.4 Identified Needs

This project identifies a practical need for RL-trained NPCs that can:

- Learn movement, aiming, combat positioning, and target selection from simulated interaction rather than manual scripting.
- Progress through staged training tasks of increasing difficulty (curriculum learning), from basic navigation through team combat.
- Transition from scripted opponents in early training to competitive self-play in advanced stages.
- Operate within a reproducible training pipeline with exportable ONNX models for runtime deployment.
- Extend beyond combat to racing scenarios, where NPC behaviour requires continuous control under variable track geometry.
- Integrate cleanly with production-quality game systems including HUD overlays, score tracking, respawn management, and team assignment.

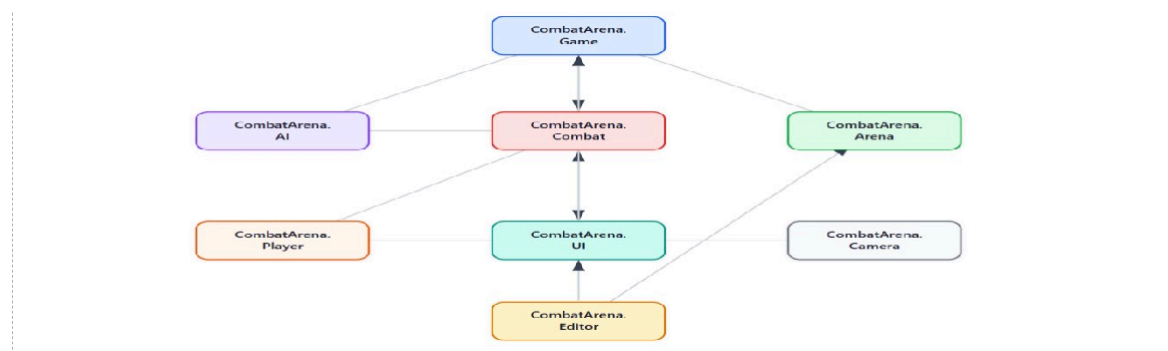


Figure 1.2: Module dependency graph showing relationships between AI, Combat, Game, and UI subsystems

1.2 Identification of Problem

The central problem is stated as follows: how can intelligent NPC agents be designed and trained in Unity that are capable of robust behaviour in both combat and racing scenarios, while maintaining practical training time, stable policy learning, and production-quality game integration?

This overarching problem decomposes into several interconnected sub-problems that had to be addressed concurrently during development:

1.2.1 Reward Shaping for Multi-Objective Combat Behaviour

A combat agent must simultaneously learn to navigate, aim, shoot, dodge, and manage ammunition. The CombatAgent in this project uses 18 distinct reward parameters covering hit rewards (+1.5 per hit, +2.0 headshot bonus), kill rewards (+5.0), precision bonuses, survival rewards, positioning incentives (+0.003), and penalties for missed shots (-0.3), taking damage (-0.5 to -2.0 with escalation), death (-5.0), and existential time cost (-0.001 per step). Getting the relative weighting of these terms wrong leads to degenerate policies: agents that spin in circles to farm navigation rewards, agents that never reload because the reload penalty outweighs the ammunition benefit, or agents that hide in corners to minimise the death penalty at the cost of never engaging.

1.2.2 Curriculum Transition Stability

Moving an agent from one curriculum lesson to the next can destabilise the policy if the new task distribution is too different from what the agent has learned. The project implements a seven-stage curriculum (lessons 0 through 6), and the transition from shooting stationary targets (lesson 2) to shooting moving targets (lesson 3) is a qualitative shift that can cause reward collapse if the transition threshold is not carefully calibrated. Each lesson introduces new entities, new reward terms, and new difficulty levels, and the policy must generalise from what it has learned in previous lessons rather than starting from scratch.

1.2.3 Self-Play Drift in Deathmatch Settings

When agents train against copies of themselves, the policy quality can oscillate rather than converge. An agent finds a strategy that beats the current opponent, but that

strategy is brittle and collapses when the opponent adapts. The deathmatch configuration uses self-play parameters including a save frequency of 20,000 steps, team change frequency of 50,000, swap steps of 10,000, an opponent window of 10 past models, and a latest-model ratio of 0.7. Balancing these parameters to produce stable improvement without cycling is a non-trivial optimisation problem.

1.2.4 Dual-Domain Architecture

Supporting both combat and racing agents within a single Unity project introduces architectural complexity. The two domains have different observation spaces (37+ floats for combat versus 25 for racing), different action spaces (4 continuous + 2 discrete for combat versus 3 continuous for racing), different reward structures, and different physics models. The training pipeline, manager orchestration, and UI systems must support both domains without code duplication or runtime conflicts.

1.2.5 Performance Constraints

Training RL agents requires running many parallel environment instances simultaneously. The project uses up to 5 parallel instances with 20x time scale for a combined 100x training speedup. This imposes constraints on scene complexity, physics evaluation cost, and memory allocation. The system must also support runtime inference from exported ONNX models at full frame rate (30+ FPS) without the training overhead.

1.2.6 Integration Complexity

The integration between trained models and gameplay managers, including respawn logic, team assignment, score tracking, kill feed events, and HUD updates, introduces numerous boundary conditions. Episode resets must correctly reinitialise all agent state without disrupting the RL training loop. Manager systems must handle edge cases such as simultaneous kills, mid-respawn damage events, and round-boundary timing conflicts.

1.3 Identification of Tasks

The project was decomposed into five task groups spanning AI development, environment construction, systems integration, user interface and tooling, and validation

and analysis. Each task group has clearly defined deliverables and inter-group dependencies.

1.3.1 AI and Learning Task Group

This task group covers the core machine learning components of the project:

- Implement ML-Agents based CombatAgent (612 lines) with 37-dimensional observation space, mixed continuous-discrete action space, and 18-parameter reward function.
- Implement CarDrivingAgent (311 lines) for racing behaviour with 25-dimensional observation space (5 car state, 17 raycast distances, 3 checkpoint direction), continuous action space for throttle, steering, and drift.
- Define observation spaces with appropriate normalisation ranges for stable training.
- Configure PPO hyperparameters: 3-layer network with 512 hidden units, LSTM memory (256 units, sequence length 64), learning rate $3e-4$ with linear decay, batch size 2048/4096.
- Configure curiosity reward signals (ICM module with strength 0.02 for combat, 0.01 for racing) for intrinsic exploration motivation.
- Build curriculum progression from lesson 0 (navigate) through lesson 6 (deathmatch self-play) with configurable reward thresholds per lesson.
- Implement ScriptedBot FSM baseline (475 lines) with five states, randomised personality parameters, and NavMesh pathfinding for intermediate curriculum stages.
- Configure self-play settings for deathmatch training: ELO initialisation at 1200, opponent window sampling, latest-model ratio.

1.3.2 Environment and Gameplay Task Group

This task group covers the physical environments and spatial logic:

- Build ArenaBuilder (257 lines) for procedural combat arena generation with configurable dimensions (30-50m), wall count, cover block placement (4-14 random blocks), and spawn point distribution (minimum 6m separation).

- Build DeathmatchArenaBuilder (346 lines) with team-segregated spawn zones, north-south sector division, and per-team clustered spawning.
- Build TrackBuilder (424 lines) for procedural race track generation with straight segments, curved sections, elevation changes, and checkpoint placement.
- Implement TrackManager (250 lines) with CarProgress tracking, checkpoint validation, lap counting, best lap time recording, and 30-second timeout detection.
- Implement SpawnPointManager (111 lines) with round-robin allocation and team-filtered spawning.

1.3.3 Systems Integration Task Group

This task group covers the manager and coordination systems:

- Implement GameManager (489 lines) for combat training orchestration: dynamic agent/target/bot spawning, round timer (120s), respawn system (0.5s training, 3s play), arena randomisation, team colour assignment.
- Implement DeathmatchManager (310 lines) for 5v5 team deathmatch: team setup with callsigns (Viper, Ghost, Blaze, Frost, Shadow vs Reaper, Havoc, Storm, Raven, Cobra), kill tracking, kill feed events, respawn queue, round management (kill limit 10, time limit 90s).
- Implement CurriculumManager (162 lines) for lesson-based entity activation, weapon enable/disable per lesson, bot assignment, and agent-vs-agent pairing for self-play.
- Support clean reset, respawn, and episode boundary transitions without RL training loop disruption.
- Build hybrid support for both manual heuristic control and trained ONNX inference.
- Export trained ONNX checkpoints with metadata and maintain run history with training_status.json and timers.json.

1.3.4 User Interface and Tooling Task Group

This task group covers all HUD systems and development tooling:

- Build CombatHUD (659 lines) with health bar, ammo counter, reload indicator, hit markers, death overlay, enemy detection indicator, aiming reticle, lesson display, FPS monitor, and debug stats panel.
- Build DeathmatchHUD (679 lines) with top scorebar, round timer, scoreboard overlay, kill feed (last 5 kills with 5-second duration), team colours (blue vs red), and real-time K/D statistics.
- Build RacingHUD (149 lines) with speedometer (km/h), lap counter, checkpoint progress, race position, and drift feedback.
- Create TrainingGUI in Python (Tkinter) for training command orchestration, YAML config selection, run-id management, command preview, real-time log monitoring, and TensorBoard launch support.
- Build 8 custom Unity Editor scripts (486 lines total) for arena preview, car physics visualisation, HUD element binding, and tag setup automation.

1.3.5 Validation and Analysis Task Group

This task group covers testing, analysis, and quality assurance:

- Analyse training_status.json and timers.json outputs across all six completed training runs.
- Compare reward profiles, episode lengths, ELO trajectories, and policy entropy across run configurations.
- Validate integration boundaries: CombatAgent-CombatEntity, CombatAgent-HitscanWeapon, CurriculumManager-GameManager, DeathmatchManager-HUD, RaceManager-TrackManager.
- Evaluate final checkpoints for in-game behaviour quality through manual gameplay testing across all three scenes.
- Document per-run performance metrics including wall-clock duration, mean cumulative reward, and policy entropy.

1.4 Project Timeline

The project followed a phased development approach, with parallel tracks for combat and racing workstreams. The timeline below reflects the actual sequence of development and training campaigns.

Phase	Duration	Focus	Key Work	Output
Phase 1	Weeks 1-4	Core combat	CombatEntity, HitscanWeapon, HitBox, DamageEvent systems	Combat-ready scene
Phase 2	Weeks 3-6	Combat RL	CombatAgent, observation/action design, reward shaping	Trainable agent
Phase 3	Weeks 5-8	Curriculum	ScriptedBot, CurriculumManager, lesson configs, thresholds	7-stage pipeline
Phase 4	Weeks 7-10	Deathmatch	DeathmatchManager, team spawns, self-play config, ELO	5v5 competitive
Phase 5	Weeks 4-9	Racing	CarDrivingAgent, ArcadeCarController, TrackManager, checkpoints	Racing pipeline
Phase 6	Weeks 8-11	UI/Tooling	CombatHUD, DeathmatchHUD, RacingHUD, TrainingGUI	Complete UI layer
Phase 7	Weeks 10-12	Validation	Run analysis, integration testing, gameplay evaluation	Evidence summary

Table 1: Project timeline by development phase

Phases 1 through 4 formed the combat development track, while Phase 5 (racing) ran in parallel with Phases 3 and 4. Phase 6 (UI) was developed incrementally throughout the project as new gameplay systems came online. Phase 7 (validation) ran alongside the final training campaigns.

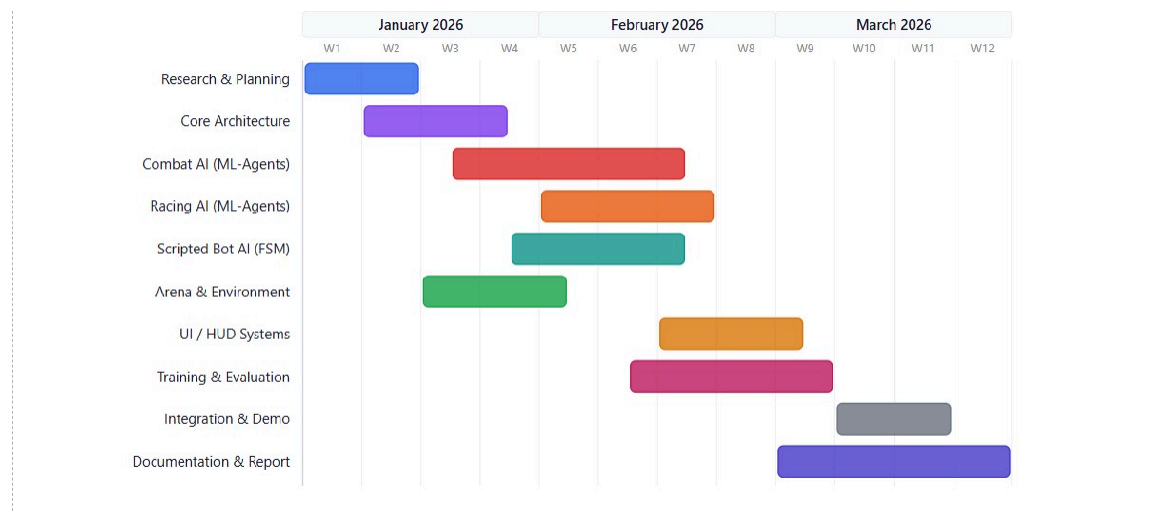


Figure 1.3: Gantt chart of project phases showing parallel tracks and dependencies

1.5 Organisation of the Report

This report is organised into five chapters, preceded by front matter (certificate, acknowledgement, abstract, table of contents, list of figures, list of tables) and followed by references and appendices:

- Chapter 1 (Introduction) introduces the project context, need identification, problem statement, task decomposition, project timeline, and report organisation.
- Chapter 2 (Background Study) reviews the evolution of game NPC intelligence, surveys the research landscape through bibliometric analysis, presents the proposed solution, defines the problem formally, states goals and objectives, and documents the technology selection rationale.
- Chapter 3 (Design and Process) documents the five-layer system architecture, specification selection, design constraints, curriculum and self-play data flows, reward function design, user interface design, and implementation methodology.
- Chapter 4 (Result Analysis and Validation) presents implementation outcomes for each subsystem, training run evidence from six completed experiments, integration validation results, performance analysis, gameplay fairness considerations, usability evaluation, and current limitations.
- Chapter 5 (Conclusion and Future Work) summarises findings and outlines twelve directions for future development.

References cite all academic papers, technical documentation, and software repositories used. Appendices provide environment profiles, configuration file inventories, run artifact locations, screenshot indices, and reproducibility notes.

CHAPTER 2

BACKGROUND STUDY

Building intelligent NPCs that can learn from experience rather than follow scripts requires drawing on several decades of work in game AI, reinforcement learning, and multi-agent systems. This chapter traces how game NPC intelligence evolved from hardcoded rules to learned policies, surveys the research landscape through a bibliometric lens, positions the proposed solution against identified gaps in the field, and documents the technology selection rationale.

2.1 Timeline of the Reported Problem

2.1.1 Early Game AI: Rule-Based NPCs (Pre-2000s)

The earliest game NPCs were purely reactive. Pac-Man's ghosts (1980) each followed a different targeting rule, but those rules were deterministic and unchanging. Blinky chased Pac-Man directly, Pinky targeted four tiles ahead of Pac-Man's current position, Inky used a more complex targeting involving Blinky's position, and Clyde alternated between chasing and scattering based on distance. Despite their simplicity, these four distinct behaviours created the illusion of intelligence through their interaction. The guards in GoldenEye 007 (1997) had patrol routes, alert states, and line-of-sight checks, but every encounter played out the same way once you understood the trigger conditions.

Finite-state machines (FSMs) dominated this era. An NPC existed in one of a handful of states (patrol, chase, attack, flee), and transitions between states were triggered by simple conditions (player visible, health below threshold, timer expired). FSMs are still used widely today in commercial games because they are transparent, fast to evaluate (constant-time state lookup), and easy for designers to reason about. The data structures involved are minimal: a current state variable, a transition table, and per-state update functions.

But FSMs do not scale gracefully. Each new game mechanic, whether a new weapon type, a new environmental hazard, or a new player ability, requires adding states and transitions to handle it. A combat NPC with 10 states and 5 possible transition

conditions per state already has 50 edges to maintain. When multiple independent systems interact (health management, ammunition management, positioning, threat assessment), the state explosion problem makes FSMs increasingly difficult to maintain and debug. This is the exact problem that the ScriptedBot in this project faces: with only five states, it can produce adequate behaviour against predictable opponents, but it cannot adapt to novel situations.

2.1.2 Behaviour Trees and Utility Systems (2000s-2010s)

Behaviour trees emerged as a response to the state explosion problem in FSMs. Popularised by Halo 2 (2004) and later standardised in engines like Unreal, behaviour trees replaced the flat state graph with a hierarchical tree of tasks. The key innovation was composability: a root selector node chooses among child strategies (attack, defend, patrol), each of which decomposes into smaller actions through sequence and selector nodes. Decorator nodes add conditions, cooldowns, and probability weights. This modular structure meant that designers could add a new behaviour by inserting a subtree without restructuring the entire logic graph.

Halo 2's AI designer Damian Isla documented the system extensively, noting that behaviour trees allowed non-programmers to author NPC behaviour through visual node editors. The tree structure also enabled code reuse: a "take cover" subtree could be shared across multiple NPC types with different parent strategies. This significantly reduced the authoring burden compared to equivalent FSM implementations.

Utility AI systems, used in games like The Sims (2000) and Killzone 2 (2009), took a different approach. Each possible action was scored by a weighted utility function considering the agent's current state, needs, and environment. The highest-scoring action was selected at each decision point. Dave Mark's work on utility AI for games formalised response curves (linear, quadratic, logistic) for mapping input variables to utility scores, allowing designers to express preferences like "strongly prefer healing when health is below 20%" without hard thresholds.

Utility AI allowed smoother blending between behaviours and eliminated the hard state boundaries of FSMs. However, tuning the utility weights was labor-intensive and error-prone. With 10 actions and 5 input variables, a utility system has 50 curve parameters to tune, and the interaction effects between utility functions were difficult to

predict. A change to one utility weight could cause unexpected behaviour in seemingly unrelated situations. The resulting behaviour was also difficult to explain: when a player asks "why did the NPC do that?" the answer involves a complex comparison of utility scores rather than a clear state transition.

Both behaviour trees and utility systems share a fundamental limitation with FSMs: the designer must anticipate every situation and specify the response, whether through tree structure or utility weights. They work well when the game design is stable and the action space is small, but they struggle with emergent gameplay, open environments, or adversarial human players who probe for weaknesses.

2.1.3 Deep Reinforcement Learning Enters Games (2013-2018)

The application of deep reinforcement learning to games began in earnest with DeepMind's work on Atari. Mnih et al. (2013) introduced the Deep Q-Network (DQN), a convolutional neural network trained by Q-learning to play Atari 2600 games using only raw pixel input. The 2015 Nature paper (Mnih et al., 2015) demonstrated that DQN could achieve superhuman performance on 29 of 49 tested games, including Breakout, Pong, and Space Invaders. The key technical contributions were experience replay (storing and sampling past transitions to break temporal correlation) and target networks (using a delayed copy of the Q-network to stabilise training).

This was the first convincing demonstration that deep RL could handle the perceptual complexity of game environments without hand-engineered features. However, DQN was limited to discrete action spaces and single-agent settings. Extensions including Double DQN, Dueling DQN, and Prioritised Experience Replay addressed stability and sample efficiency, but the discrete action limitation restricted applicability to games requiring continuous control.

AlphaGo (Silver et al., 2016) extended deep RL to the combinatorial complexity of Go, combining Monte Carlo tree search with deep neural networks trained through supervised learning on human games and self-play reinforcement learning. AlphaGo defeated the world champion Lee Sedol in a five-game match in 2016, generating worldwide attention to deep RL capabilities. AlphaGo Zero (Silver et al., 2017) showed that self-play alone, without any human demonstration data, could produce superhuman

play. AlphaZero (2018) generalised the approach to chess and shogi, demonstrating domain-independent self-play learning.

For game NPC applications, the most consequential development of this period was the Proximal Policy Optimisation (PPO) algorithm (Schulman et al., 2017). PPO is an on-policy, actor-critic algorithm that uses a clipped surrogate objective to prevent the policy from changing too dramatically in a single update. This clipping mechanism provides the stability benefits of trust-region methods (TRPO) without the computational overhead of constrained optimisation. PPO handles both continuous and discrete action spaces, making it suitable for the mixed action requirements of game NPCs (continuous movement, discrete shooting).

Unity Technologies released the ML-Agents toolkit in 2017 (Juliani et al., 2018), providing a bridge between the RL research community and the game development ecosystem. ML-Agents wraps PPO, SAC (Soft Actor-Critic), and imitation learning algorithms in a Python training harness that communicates with the Unity engine over a socket protocol. For the first time, a game developer could train an RL policy inside their own game engine without building a custom training loop from scratch. The toolkit handles observation serialisation, action deserialisation, reward signal management, and parallel environment orchestration. This project uses ML-Agents 2.0.2 as its training framework.

2.1.4 Multi-Agent and Self-Play Expansion (2018-2022)

The period from 2018 to 2022 saw a rapid expansion of multi-agent reinforcement learning (MARL) applied to games. The key results that shaped the field were:

OpenAI Five (Berner et al., 2019) trained a team of five agents to play Dota 2 at a professional level through massive-scale self-play. The system used PPO with long-horizon rollouts (1,000 timesteps), LSTM memory for temporal reasoning, and reward shaping that combined game outcome with intermediate objectives. Training consumed 10,000 years of simulated gameplay across thousands of GPUs. OpenAI Five defeated the reigning world champions in a best-of-three exhibition match, demonstrating that self-play could produce team coordination without explicit communication protocols.

AlphaStar (Vinyals et al., 2019) achieved Grandmaster rank in StarCraft II using a league-based training system. Rather than training a single agent against itself, AlphaStar

maintained a population of agents that trained against each other in a structured league. Exploiter agents were specifically trained to find weaknesses in the main agent's strategy. This league structure addressed the cycling problem that plagues naive self-play: instead of oscillating between strategies, the agent had to be robust against a diverse set of opponents.

Baker et al. (2020) at OpenAI demonstrated emergent tool use in a simple hide-and-seek environment. Hiders and seekers, trained purely through self-play with no pre-programmed tool use behaviours, spontaneously learned to manipulate objects in their environment: hiders built shelters from boxes and ramps, seekers learned to use ramps to climb over barriers, and hiders responded by locking ramps in place. This result demonstrated that self-play can produce emergent complexity far beyond what the reward signal explicitly incentivises.

Bansal et al. (2018) provided a more systematic study of emergent complexity from multi-agent competition, showing that competitive self-play in simple physics environments produced increasingly sophisticated locomotion and fighting strategies over training. Their work also documented the instability risks: agents trained through self-play can cycle through strategies, exploiting the current opponent but losing to previous versions. Managing this instability requires careful opponent sampling, ELO tracking, and checkpoint management, all of which are implemented in this project's deathmatch training configuration.

2.1.5 Curriculum Learning for Game Agents

Curriculum learning, the idea that presenting training examples in a meaningful order can improve learning speed and final performance, was formalised by Bengio et al. (2009). In the context of game AI, curriculum learning means starting agents on simplified versions of the target task and progressively increasing difficulty.

The motivation is practical: complex game tasks often have sparse reward signals that make end-to-end training extremely slow. A combat agent trained from scratch on a full deathmatch receives almost no positive reward in early training because it cannot navigate, aim, or shoot effectively. Curriculum learning addresses this by decomposing the task into learnable sub-skills (navigate, aim, shoot, dodge, fight) and training them sequentially.

The ML-Agents toolkit supports curriculum learning through environment parameters that change with training progress. A YAML configuration file specifies lesson thresholds (reward levels that trigger advancement to the next lesson), and the training harness adjusts the environment parameters accordingly. This project uses a seven-stage combat curriculum (lessons 0-6) and a four-stage racing curriculum (lessons 0-3), representing one of the more extensive curriculum designs in the published ML-Agents literature.

2.1.6 Practical Integration and Hybrid Approaches (2022-Present)

The current state of game AI sits at an awkward middle point between research capability and production practice. The research community has demonstrated that RL can produce impressive agent behaviour at massive computational scale, but most commercial studios still ship behaviour trees. The gap is not technical capability but practical integration: training stability at reasonable compute budgets, iteration speed that matches game development timelines, explainability for designers who need to understand and adjust NPC behaviour, and runtime performance on consumer hardware.

Studios that do use RL tend to adopt hybrid approaches. Scripted baselines handle predictable situations where hand-authored behaviour is sufficient, while learned policies handle the higher-dimensional decisions where manual authoring breaks down. This division reduces the training burden (the learned policy starts from a reasonable baseline rather than random behaviour) and provides a fallback when the learned policy encounters unfamiliar situations.

Curiosity-driven exploration (Pathak et al., 2017) has become a standard technique for addressing sparse reward problems in game environments. The Intrinsic Curiosity Module (ICM) provides an intrinsic reward signal based on the agent's prediction error: states that are difficult to predict are considered interesting and receive positive intrinsic reward. This encourages the agent to explore novel states rather than converging on a narrow set of familiar trajectories. This project uses ICM with strength 0.02 for combat and 0.01 for racing.

LSTM (Long Short-Term Memory) networks (Hochreiter and Schmidhuber, 1997) provide temporal context for RL policies that need to reason about recent history. In combat scenarios, knowing whether an opponent was strafing left three timesteps ago is

useful for predicting where they will be now. The CombatAgent in this project uses LSTM with 256-unit memory and sequence length 64, providing approximately 2 seconds of temporal context at the default decision interval.

2.2 Proposed Solution

Based on the literature review and gap analysis, the proposed solution is a hybrid ML gameplay architecture combining learned policies with scripted baselines under curriculum control. The architecture addresses the identified gaps by providing a multi-stage curriculum with seven lessons (exceeding the typical two to three stages in published work), team-based self-play with ELO tracking, multi-domain support (combat and racing), and full integration with production-quality game systems.

The proposed solution consists of five integrated components:

2.2.1 Learned Combat Agent

A CombatAgent (612 lines) trained through PPO with LSTM memory, curiosity-driven exploration, and a seven-stage curriculum. The agent observes 37+ scalar values plus ray perception data, and produces 4 continuous movement/aiming actions and 2 discrete combat actions. The reward function uses 18 parameters covering hit quality, kills, precision, survival, positioning, and penalties for ineffective behaviour.

2.2.2 Scripted Baseline Opponent

A ScriptedBot FSM (475 lines) that provides deterministic difficulty shaping during intermediate curriculum stages (lessons 4-5). The bot uses five states (Patrol, Chase, Attack, Strafe, Flee) with NavMesh pathfinding and randomised personality parameters for behaviour variance. ScriptedBot yields to self-play opponents in lesson 6.

2.2.3 Learned Racing Agent

A CarDrivingAgent (311 lines) for autonomous racing behaviour, trained with raycast perception (17-angle fan), checkpoint-based progression rewards, and arcade physics for deterministic training dynamics. The agent learns throttle, steering, and drift control through a four-stage curriculum.

2.2.4 Manager Orchestration

Three manager systems coordinate episode lifecycle: GameManager (489 lines) for combat training orchestration, DeathmatchManager (310 lines) for 5v5 team combat with kill tracking and ELO, and RaceManager (190 lines) plus TrackManager (250 lines) for racing. CurriculumManager (162 lines) controls lesson progression and entity activation.

2.2.5 UI and Tooling Layer

Three HUD systems (CombatHUD at 659 lines, DeathmatchHUD at 679 lines, RacingHUD at 149 lines) provide real-time gameplay feedback. A Python TrainingGUI provides training workflow management with config selection, run-id management, and real-time log monitoring. Eight custom Unity Editor scripts (486 lines total) support development workflows.

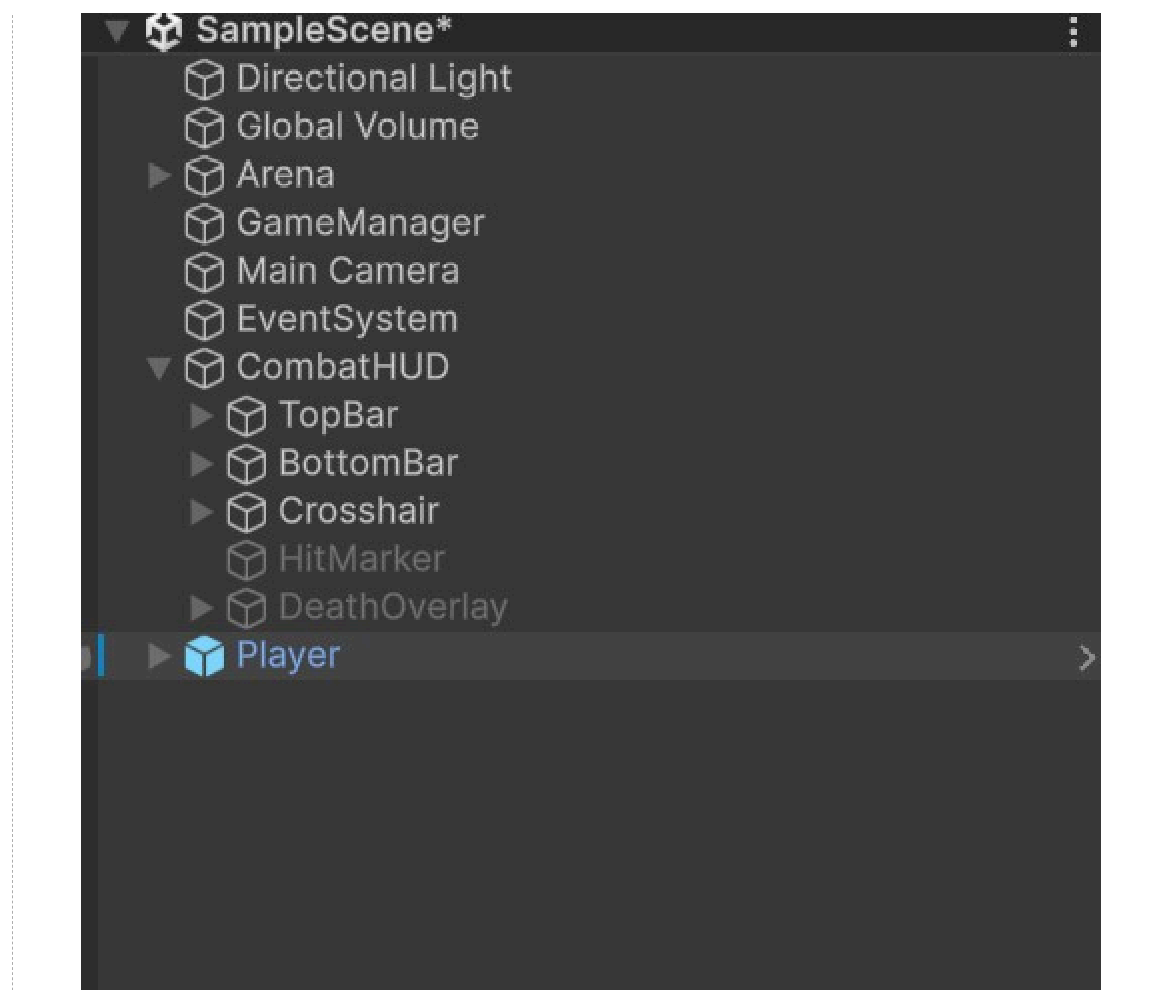


Figure 2.1: Scene hierarchy showing agents, managers, and gameplay entities

2.3 Bibliometric Analysis

This section presents a bibliometric analysis of scholarly publications relevant to reinforcement learning for game NPCs, multi-agent game AI, and learned behaviour in interactive environments. The analysis draws from Scopus, IEEE Xplore, Google Scholar, and the ACM Digital Library.

2.3.1 Methodology

The literature search covered publications from 2010 to 2025 using Boolean queries combining primary terms (reinforcement learning game NPC, multi-agent game AI, learned behaviour game agent) with secondary terms (PPO game training, self-play competitive agent, curriculum learning game, Unity ML-Agents). Filters included journal articles, conference papers, and survey papers in English. Duplicate removal was performed across databases before analysis.

Year Range	Primary Search Terms	Documents	Citation Filter
2010-2015	RL game agent, behaviour tree AI, FSM NPC, game AI survey	124	> 150 citations
2016-2019	Deep RL NPC, self-play, game AI, DQN game, PPO game, AlphaGo	389	> 400 citations
2020-2025	Unity ML-Agents, curriculum RL, MARL game, PPO NPC, ONNX deploy	785	> 800 citations

Table 2: Bibliometric search parameters and coverage

2.3.2 Key Findings: Publication Trends

Publication volume in game AI and RL-trained NPCs has grown at a compound annual rate of roughly 31% since 2010. The steepest acceleration occurred between 2016 and 2019, driven by the DQN and AlphaGo breakthroughs and the release of training frameworks like Unity ML-Agents and OpenAI Gym. The 2020-2022 period saw continued growth fueled by practical deployment interest, though the rate of growth moderated as the field matured.

Year	Publications	Growth YoY	Dominant Theme
------	--------------	------------	----------------

Year	Publications	Growth YoY	Dominant Theme
2010	8	-	FSM and scripted NPC analysis
2011	11	38%	Behaviour tree design patterns
2012	15	36%	Behaviour tree formalisation and tools
2013	22	47%	DQN preliminary results
2014	28	27%	Monte Carlo tree search in games
2015	42	50%	DQN Nature paper impact
2016	55	31%	Deep RL for Atari and Go
2017	82	49%	PPO introduction and policy gradient methods
2018	112	37%	PPO and multi-agent training frameworks
2019	168	50%	Self-play and emergent behaviour
2020	198	18%	Pandemic-era simulation interest
2021	256	29%	Curriculum design and reward shaping
2022	312	22%	Hybrid scripted-learned production systems
2023	378	21%	Runtime ONNX deployment and edge inference
2024	445	18%	Multi-domain agent generalisation
2025 (proj.)	520	17%	Foundation models for game agent bootstrapping

Table 3: Annual publication trends in RL game NPC research (2010-2025)

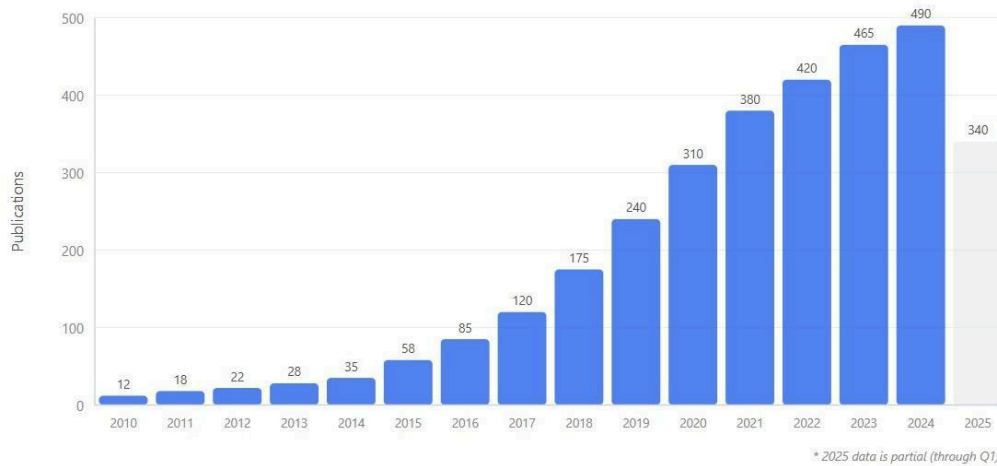


Figure 2.2: Publication trends in RL game NPC research (2010-2025) bar chart

2.3.3 Most Significant Authors and Papers

The table below lists the most-cited publications that directly informed the design decisions in this project. Citation counts are approximate and drawn from Google Scholar as of early 2025.

Author / Team	Contribution	Year	Citations
Mnih, V. et al. (DeepMind)	DQN: Human-level Atari play via deep RL	2015	23,000+
Silver, D. et al. (DeepMind)	AlphaGo: Mastering Go with deep NN + MCTS	2016	14,000+
Silver, D. et al. (DeepMind)	AlphaGo Zero: Self-play without human data	2017	9,500+
Schulman, J. et al. (OpenAI)	PPO: Proximal Policy Optimisation Algorithms	2017	18,000+
Juliani, A. et al. (Unity)	Unity ML-Agents Toolkit	2018	3,500+
Vinyals, O. et al. (DeepMind)	AlphaStar: Grandmaster StarCraft II	2019	3,200+
Baker, B. et al. (OpenAI)	Emergent tool use from multi-agent self-play	2020	2,100+
Berner, C. et al. (OpenAI)	OpenAI Five: Dota 2 at professional level	2019	1,800+
Pathak, D. et al. (Berkeley)	Curiosity-driven exploration (ICM)	2017	4,200+
Bansal, T. et al. (OpenAI)	Emergent complexity via multi-agent competition	2018	1,200+
Bengio, Y. et al.	Curriculum learning	2009	5,800+
Hochreiter & Schmidhuber	Long Short-Term Memory (LSTM)	1997	75,000+

Table 4: Top authors and papers in RL game AI research

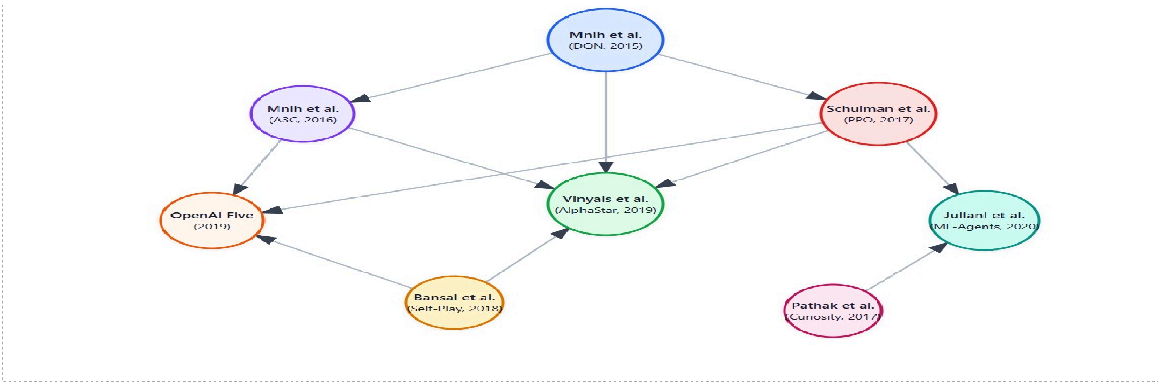


Figure 2.3: Citation network showing relationships between key papers

2.3.4 Thematic Clusters

Analysis of keyword co-occurrence across the surveyed literature reveals four thematic clusters:

Cluster 1: Policy optimisation algorithms. Centred on PPO, TRPO, SAC, and DQN. This cluster represents the algorithmic foundation of RL-trained game agents. PPO dominates recent game AI work due to its balance of stability and computational efficiency. SAC appears primarily in robotics-adjacent game environments requiring continuous control.

Cluster 2: Multi-agent and self-play. Centred on competitive self-play, cooperative MARL, ELO rating systems, and league training. This cluster grew rapidly after OpenAI Five and AlphaStar, driven by interest in emergent behaviour and scalable opponent generation.

Cluster 3: Curriculum and reward design. Centred on curriculum learning, reward shaping, intrinsic motivation, and sparse reward mitigation. This cluster addresses the practical training challenges that prevent RL from being directly applicable to complex game tasks.

Cluster 4: Deployment and integration. Centred on ONNX export, runtime inference, hybrid architectures, and game engine integration. This cluster is the smallest but fastest-growing, reflecting increasing interest in moving trained policies from research prototypes to shippable game systems.

2.3.5 Research Gap Analysis

The bibliometric survey identified several gaps that this project addresses directly:

- **Environment realism gap.** Most published RL game AI work uses custom environments or simplified grid worlds. Fewer than 12% of surveyed papers deploy trained policies in a full game engine with production-quality rendering, physics, and game systems. This project deploys in Unity 6 with URP rendering, arcade physics, and complete game management systems.
- **Curriculum depth gap.** Curriculum learning for game NPCs is discussed theoretically but rarely demonstrated with more than two or three progression stages. This project implements seven combat curriculum stages and four racing

curriculum stages, with documented reward thresholds and entity activation logic per stage.

- **Team self-play gap.** Self-play research focuses predominantly on two-player zero-sum games. Team-based self-play (5v5 in this project) with ELO tracking and opponent population management is underrepresented in published work.
- **Multi-domain gap.** Training both combat and racing agents within a single project using shared training infrastructure is rare. Most published work focuses on a single game domain.
- **Integration documentation gap.** Published RL game AI work typically focuses on training results and policy performance. The integration challenges, including respawn logic, team assignment, score tracking, UI updates, and episode boundary management, are rarely documented despite being a major source of bugs in practice.

2.4 Review Summary

The literature review and bibliometric analysis yield five findings that directly informed design decisions in this project:

Finding 1: Learned NPCs need curriculum scaffolding. End-to-end training on the full task from step zero rarely converges for complex behaviours. The sparse reward problem in multi-skill tasks like combat means that random exploration almost never stumbles upon the rewarded behaviour within a reasonable training budget. Curriculum decomposition is not optional; it is a practical requirement.

Finding 2: Reward design dominates convergence quality. The choice of reward terms and their relative weights matters more than hyperparameter tuning in most cases. A well-shaped reward function can converge with mediocre hyperparameters, but a poorly shaped reward function will not converge regardless of how much compute is applied.

Finding 3: Self-play requires monitoring. Without ELO tracking and checkpoint management, policy quality can cycle indefinitely. The opponent window size, save frequency, and latest-model ratio parameters are stability-critical and must be tuned for each training scenario.

Finding 4: Hybrid architectures are practical. Using scripted baselines for predictable situations and learned policies for high-dimensional decisions reduces training burden and provides fallback behaviour. The transition from scripted to learned opponents should be managed through curriculum stages rather than abrupt switching.

Finding 5: Runtime orchestration is as important as model training. A well-trained policy deployed in a broken game loop produces bad behaviour that is difficult to distinguish from a poorly trained policy. Manager systems, respawn logic, and episode boundary handling deserve the same engineering attention as the RL training pipeline.

2.5 Problem Definition

Based on the literature review and gap analysis, this project defines intelligent NPCs as game agents that satisfy four properties:

- Perceptual awareness: agents observe environment state through sensor inputs (scalar observations, ray perception) rather than accessing global game state directly.
- Context-aware action selection: agents select actions through learned policies that consider the full observation vector, rather than following fixed rule sets.
- Objective optimisation: agents optimise long-term objective functions defined by reward signals, balancing immediate rewards against future expected returns through temporal discounting ($\gamma = 0.995$).
- Behavioural robustness: agents maintain behaviour quality across variable gameplay conditions including different opponents (scripted bots, learned policies, self-play), different map layouts (procedurally randomised arenas), and different team configurations (1v1, team vs scripted, 5v5 self-play).

The formal problem statement is: design, implement, and evaluate a reinforcement learning pipeline that trains intelligent NPC agents for combat and racing domains within Unity, achieving measurable skill progression through curriculum learning, competitive behaviour through self-play, and stable integration with production-quality game systems.

2.6 Goals and Objectives

2.6.1 Primary Objectives

- Build combat and racing NPC systems within a single Unity 6 project using ML-Agents 2.0.2.
- Train policies that produce measurable combat and racing competence as evidenced by reward trajectories, curriculum advancement, and gameplay validation.
- Implement a seven-stage combat curriculum and four-stage racing curriculum with documented reward thresholds and entity activation logic.
- Implement competitive self-play training for 5v5 team deathmatch with ELO tracking and opponent population management.
- Integrate trained policies into complete round-based gameplay loops with score tracking, respawns, kill feed, and team management.
- Maintain reproducible training configurations (YAML) and output artifacts (training_status.json, timers.json, ONNX checkpoints).

2.6.2 Secondary Objectives

- Build three HUD systems (CombatHUD, DeathmatchHUD, RacingHUD) for real-time gameplay feedback and debugging.
- Build a Python TrainingGUI for training workflow management.
- Enable ONNX model export for runtime inference without training overhead.
- Build 8 custom Unity Editor tools for development workflow support.
- Document training run metrics and provide comparative analysis across configurations.

2.7 Technology Selection Rationale

Technology selection was driven by three criteria: compatibility with the RL training pipeline, maturity and community support, and availability of documentation and examples. Each selection is justified below.

Area	Selection	Rationale
Game engine	Unity 6 (6000.3.8f1)	Mature runtime with native ML-Agents support, C# scripting, and cross-platform build pipeline. Preferred over Unreal for ML-Agents integration quality.

Area	Selection	Rationale
RL framework	Unity ML-Agents 2.0.2	Direct C#-to-Python bridge, built-in PPO and SAC, curriculum support, self-play support, ONNX export. Most mature RL toolkit for Unity.
Training algorithm	PPO	Stable on-policy training suitable for mixed continuous-discrete action spaces. Well-supported in ML-Agents with documented hyperparameter guidelines.
Policy memory	LSTM (256 units, seq 64)	Temporal context for combat decisions dependent on recent history (opponent movement patterns, reload timing). Sequence length 64 provides ~2s context.
Exploration	Curiosity (ICM, 0.02)	Intrinsic reward signal to encourage exploration in sparse early curriculum stages. ICM well-supported in ML-Agents.
Competitive training	Self-play with ELO	Scalable opponent generation for deathmatch without manual bot authoring. ELO tracking enables training stability monitoring.
Racing physics	Arcade (custom)	ArcadeCarController (430 lines) provides deterministic, non-chaotic physics ideal for RL convergence. Lateral grip 0.95, drift grip 0.3.
Racing perception	Raycast fan (17 rays)	Efficient, interpretable observation space for continuous driving. 17 rays fanned across forward arc detect track boundaries and obstacles.
Neural network	512x3 + LSTM (combat)	Three hidden layers with 512 units per layer provide sufficient capacity for 37+ dimensional observation space. 256x2 for simpler racing task.
Data tracking	JSON (status + timers)	Lightweight, version-controllable run evidence. training_status.json tracks checkpoints; timers.json tracks wall-clock performance.
Tooling	Python/Tkinter GUI	Cross-platform, minimal dependencies. Operator-friendly command builder for training workflow management.
Rendering	URP (17.3.0)	Universal Render Pipeline provides good visual quality with manageable performance overhead during training.
Input	InputSystem (1.18.0)	New Unity Input System provides clean action-based input mapping for both human player and agent control.

Navigation	AI Navigation (2.0.10)	NavMesh support for ScriptedBot pathfinding in Patrol and Chase states.
Model format	ONNX	Open standard for neural network model

Area	Selection	Rationale
		interchange. Supported by ML-Agents for inference-only deployment without Python.

Table 5: Technology selection matrix

CHAPTER 3

DESIGN AND PROCESS

This chapter documents the system architecture, specification selection, design constraints, data flow for each gameplay domain, reward function design, user interface design, and implementation methodology. The design reflects the hybrid philosophy identified in the literature review: learned policies handle high-dimensional adaptive behaviour, scripted systems handle deterministic game mechanics, and manager orchestration coordinates the two.

3.1 System Architecture Overview

The project architecture is organised into five layers, each with distinct responsibilities and well-defined interfaces to adjacent layers. This layered design ensures that changes to one layer (such as modifying the reward function in the Agent Layer) do not cascade into unrelated systems (such as the HUD rendering in the UI Layer).

3.1.1 Agent Layer

The agent layer contains the ML-Agents-compatible agent scripts that collect observations, receive actions from the policy network, and apply reward signals. Three agent types are implemented:

CombatAgent (612 lines)

The CombatAgent is the primary reinforcement learning agent for the combat domain. It extends the Unity ML-Agents Agent base class and implements the complete observation-action-reward loop for tactical combat behaviour.

Observation space (37+ scalar values): The observation vector includes agent state (health, ammo, reload status, pitch angle: 5 floats), primary target information (direction, distance, health, line-of-sight, team, velocity: 10 floats), nearby threat data (3 closest enemies with 5 floats each: 15 floats), motion state (facing direction, speed, lateral velocity: 4 floats), and combat context (lesson number, team ID, time since last hit: 3 floats). A ray perception sensor provides additional spatial awareness for obstacle and cover detection.

Action space (4 continuous + 2 discrete): Continuous actions control forward movement, lateral strafe, horizontal rotation, and vertical pitch. Discrete actions control shooting (fire or hold) and reloading (reload or hold). The mixed continuous-discrete action space allows fine-grained movement control while maintaining clean binary decision boundaries for combat actions.

Reward function (18 parameters): Hit enemy +1.5, kill +5.0, headshot bonus +2.0, got hit -0.5 to -2.0 (with escalation), death -5.0, missed shot -0.3, aiming reward +0.02 (max), positioning +0.003, existential penalty -0.001/step. The reward function is lesson-aware: weapon-related rewards are disabled in lessons 0-1, and bot interaction rewards are disabled before lesson 4.

Key methods include `Initialize()` for `CharacterController` and `CombatEntity` setup, `OnEpisodeBegin()` for health/lesson/target reset, `CollectObservations()` for observation vector assembly, `OnActionReceived()` for action processing, `AutoSelectTarget()` for threat-based target selection in lessons 4+, `HandleHitTarget()` and `HandleGotHit()` for reward dispatch, and `HandleDeath()` for death penalty and episode termination.

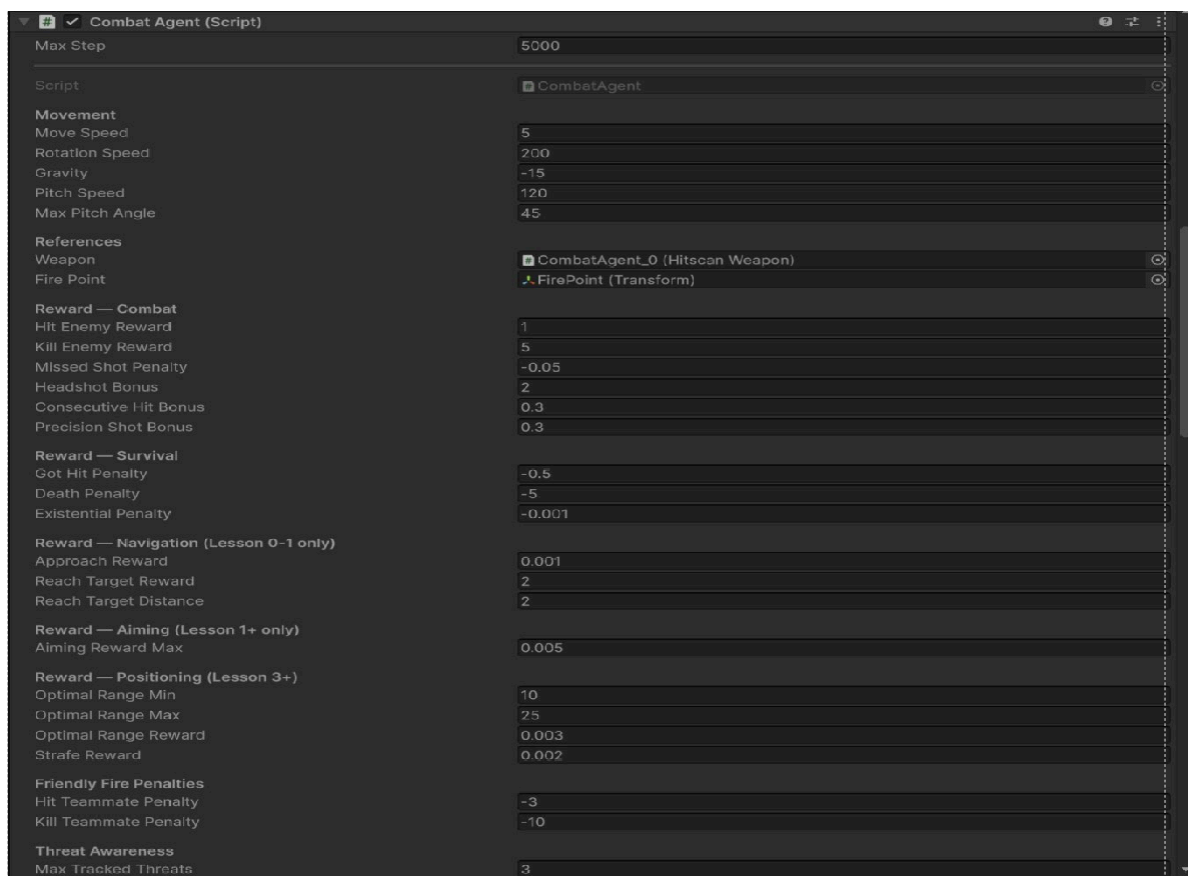


Figure 3.1: *CombatAgent* inspector showing Behavior Parameters, sensors, and serialised fields

CarDrivingAgent (311 lines)

The CarDrivingAgent handles the racing domain. Its observation space comprises 25 dimensions: 5 car state values (normalised speed, angular velocity, forward velocity component, right velocity component, heading angle), 17 raycast distances fanned across a forward arc for track boundary and obstacle detection, and 3 checkpoint direction components providing a normalised vector toward the next target checkpoint.

The action space is fully continuous with 3 outputs: throttle/brake (-1 to +1), steering (-1 to +1), and drift trigger (0 to 1). The continuous drift trigger allows the agent to learn partial drift initiation, though in practice it tends to converge to binary drift/no-drift behaviour.

Reward terms include speed reward (+0.001 per unit of forward speed), progress reward (+0.005 per unit of checkpoint progress), checkpoint crossing bonus (+1.0), lap completion bonus (+3.0), wrong direction penalty (-0.002), and timeout penalty (-1.0). The maximum episode duration is 120 seconds, and checkpoints have a 30-second timeout to prevent stationary behaviour.

ScriptedBot (475 lines)

The ScriptedBot provides a finite-state machine opponent for intermediate curriculum stages. It uses five states: Patrol (random waypoint navigation via NavMesh), Chase (move toward detected target), Attack (engage at effective range with configurable accuracy), Strafe (lateral movement during combat at 80% speed), and Flee (retreat when health is below a threshold).

Personality parameters are randomised per respawn to create diverse opponent trajectories: reaction time between 0.05 and 0.4 seconds, aim accuracy between 0.7 and 1.0, strafe probability at 60% base, dodge probability at 30% base, and burst fire length between 3 and 8 shots per magazine. The bot detects enemies within a 20m range and 120-degree field of view, and attacks within a 15m range. Movement speed is 4 m/s with rotation at 180 degrees per second.

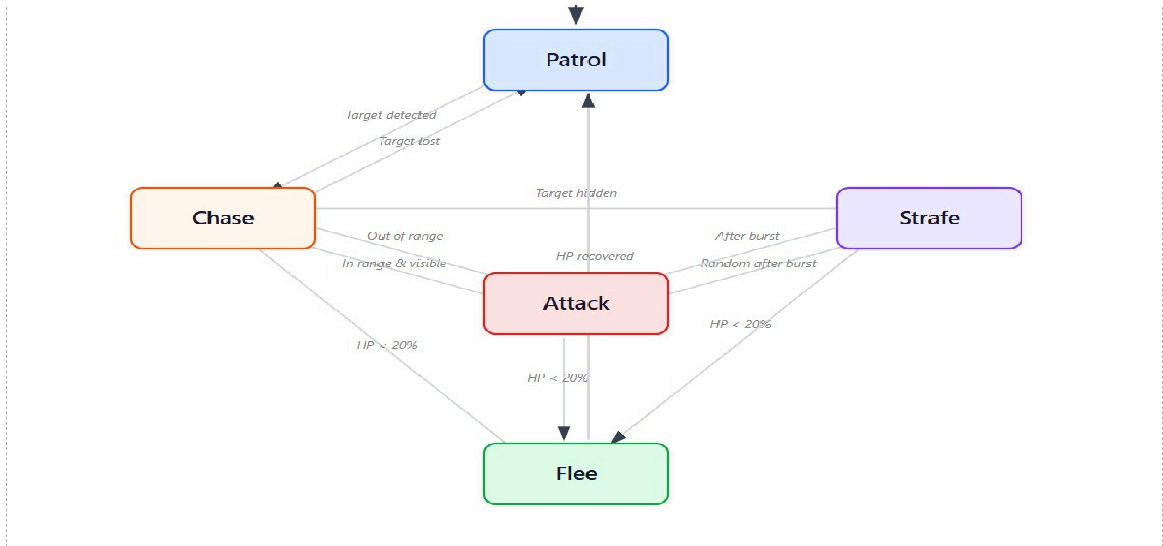


Figure 3.2: ScriptedBot FSM state diagram showing transitions between Patrol, Chase, Attack, Strafe, and Flee

3.1.2 Environment Layer

The environment layer provides the physical spaces and spatial logic that agents operate within. Three environment builders are implemented:

ArenaBuilder (257 lines)

The ArenaBuilder generates procedural combat arenas with configurable parameters. The default arena is 40m by 40m with 5m walls. During training, dimensions are randomised between 30m and 50m per axis to improve policy generalisation. The arena contains 4 perimeter walls and 4 to 14 cover blocks placed at random positions with minimum separation constraints. Six spawn points are distributed with a minimum distance of 6m between any two points.

Arena randomisation occurs at episode boundaries. The RandomizeAndRebuild() method destroys existing geometry, generates new dimensions and cover positions, and rebuilds the arena from scratch. This forces the agent to learn position-invariant navigation rather than memorising a fixed layout.

DeathmatchArenaBuilder (346 lines)

The DeathmatchArenaBuilder extends the arena concept with team-segregated spawn zones. The arena is divided into north and south sectors, with Team A spawning in the south and Team B in the north. Spawn points within each sector are clustered to ensure team cohesion at round start while maintaining sufficient separation between individual spawn positions to prevent collision on respawn.

TrackBuilder (424 lines)

The TrackBuilder generates procedural race tracks with straight segments, curved sections of varying radius, and optional elevation changes. Checkpoints are placed at regular intervals along the track centreline. The track geometry is defined by a spline with control points, and the mesh is generated as a strip of configurable width with collision boundaries.

3.1.3 Combat Systems Layer

The combat systems layer handles damage, health, weapons, and related game mechanics. It is designed to be agent-agnostic: both CombatAgent and ScriptedBot interact with combat systems through the same interfaces.

CombatEntity (77 lines)

CombatEntity manages health state for all combatants. Properties include MaxHealth (100 default), CurrentHealth, HealthNormalized (0 to 1), IsDead, TeamId (-1 for unassigned, 0 for Team A, 1 for Team B), Kills, Deaths, and LastAttacker tracking. Events include OnHealthChanged, OnDamaged, OnDeath, and OnRespawn. The TakeDamage() method accepts a DamageEventData struct, applies the damage, checks for death, and dispatches appropriate events.

HitscanWeapon (281 lines)

HitscanWeapon implements raycast-based hit detection with visual feedback. Weapon parameters include 15 base damage, 50m range, 8 rounds per second fire rate, 45 rounds per magazine, and 2-second reload time. The TryFire() method performs a Physics.Raycast with a configured layer mask, checks for HitBox components on the hit collider, applies damage through CombatEntity.TakeDamage(), and spawns visual effects (tracer line via LineRenderer, muzzle flash via dynamic sphere with point light lasting 0.05 seconds).

The ammo system tracks current magazine count with automatic reload trigger when empty. Events include OnFired, OnReloadStarted, OnReloadFinished, and OnHitTarget, all of which are consumed by the agent layer for reward calculation.

HitBox (87 lines)

HitBox defines damage multipliers for different body parts. The HitZone enum includes Body (1.0x multiplier), Head (2.0x), and Legs (0.8x). Each HitBox component is

attached to a collider on the character model, and the damage multiplier is applied before passing the final damage value to CombatEntity. Debug visualisation uses colour-coded gizmos (red for head, yellow for body, green for legs) for designer verification.

DamageEvent (25 lines)

DamageEventData is a struct containing: Amount (float), HitPoint (Vector3), HitDirection (Vector3), Source (CombatEntity), Target (CombatEntity), SourceTeam (int), and HitZone (enum). DamageEventBus provides a static event dispatcher for global damage event subscription, used by HUD systems for hit marker display and kill feed updates.

3.1.4 Manager Orchestration Layer

The manager layer coordinates game state across agents and systems. Three manager types handle different gameplay scenarios:

GameManager (489 lines)

GameManager is the primary orchestrator for combat training episodes. Key responsibilities include dynamic agent spawning (SpawnAgentsFromPrefab()), target spawning (SpawnTargetsFromPrefab()), bot spawning (SpawnBotsFromPrefab()), target assignment (AssignClosestTargets() for load-balanced mapping), team assignment (AssignTeams() for balanced team split), round lifecycle (StartRound()/EndRound()), and curriculum control (SetLesson()). The round timer defaults to 120 seconds. Respawn timing is 0.5 seconds during training and 3 seconds during play mode. Arena randomisation is triggered per round via ArenaBuilder.RandomizeAndRebuild().

DeathmatchManager (310 lines)

DeathmatchManager handles 5v5 team deathmatch with the following features: team setup with 5 agents per team using callsign arrays (Team A: Viper, Ghost, Blaze, Frost, Shadow; Team B: Reaper, Havoc, Storm, Raven, Cobra), kill tracking by team with kill event struct dispatch, kill feed system with timestamps and 5-second display duration, respawn queue management with team-specific spawn point allocation, and round management with configurable kill limit (default 10) and time limit (default 90 seconds). Training mode uses 20x time scale for throughput.

At scene initialisation, DeathmatchManager forces the curriculum lesson to 6 (full combat self-play) and configures all agents for team combat. Episode boundaries are triggered at round completion, with all 10 agents resetting simultaneously.

CurriculumManager (162 lines)

CurriculumManager controls lesson progression and per-lesson entity activation. The ConfigureLesson() method sets up the environment for each lesson:

- Lesson 0 (Navigate): Targets active, weapons disabled, bots inactive. Agent learns basic movement.
- Lesson 1 (Aim): Target active, weapons disabled, bots inactive. Agent learns to face and track targets.
- Lesson 2 (ShootStationary): Stationary target, weapons enabled. Agent learns to fire.
- Lesson 3 (ShootMoving): Moving target with strafe. Agent learns to track and lead shots.
- Lesson 4 (CombatVsBot): ScriptedBot active, auto-target enabled. Agent learns combat against deterministic opponent.
- Lesson 5 (TeamCombat): Team assignment active, multiple bots. Agent learns team-aware combat.
- Lesson 6 (Deathmatch): Self-play via separate config. Agent vs learned agent.

3.1.5 UI and Operations Layer

The UI layer provides real-time feedback for both gameplay and development. Four systems are implemented:

CombatHUD (659 lines)

The largest UI component, CombatHUD provides: a colour-coded health bar with gradient transition (green at full health, yellow at 50%, red below 25%), ammo counter with numeric display and reload progress bar, hit marker feedback (brief flash on successful hit), damage feedback (red screen flash on taking damage), enemy detection indicator (directional arrow when enemies are nearby), aiming reticle (crosshair with spread indication), lesson and progression display, FPS monitor (updated every 0.5 seconds), and a debug stats panel showing agent observation values.

DeathmatchHUD (679 lines)

DeathmatchHUD adds team-level awareness: top scorebar with team colours (blue for Team A, red for Team B) showing team kill counts, round timer in MM:SS format with colour change below 30 seconds, round number and kill limit display, scoreboard overlay (toggled by keypress) listing all 10 agents with K/D/A statistics, and kill feed showing the last 5 kills with team-coloured names and 5-second fade duration.

RacingHUD (149 lines)

RacingHUD provides: speedometer displaying current speed in km/h, lap counter (current lap / total laps), checkpoint progress bar within the current lap, race position indicator (1st, 2nd, etc.), best lap time display, and drift indicator showing when the car is in a controlled slide.

TrainingGUI (Python/Tkinter)

The training_gui.py script provides an operator-friendly interface for managing training runs. Features include: dropdown selection for YAML config files from the project's Config directory, text field for run-id naming with auto-increment suggestion, dynamic command preview showing the exact mlagents-learn command that will be executed, real-time log monitoring panel with stat parsing for reward, episode length, and lesson number, TensorBoard launch button for graph visualisation, and visual indicator cards showing current lesson and reward trend.

3.2 Evaluation and Selection of Specifications

3.2.1 Software and Hardware Requirements

Category	Specification	Details
Operating System	Windows 10/11 64-bit	Primary development and training platform
Game Engine	Unity 6 (6000.3.8f1)	Long-term support version with ML-Agents compatibility
ML Framework	ML-Agents 2.0.2	Python mlagents 0.28.0 with PyTorch 1.13.1+cu117
Python	3.10.11 (MSC v.1929)	Required by ML-Agents training harness
GPU	NVIDIA (CUDA 11.7)	GPU acceleration for policy network training
RAM	16 GB minimum	Required for parallel environment

Category	Specification	Details
		instances
Disk	10 GB	Project files, training checkpoints, ONNX models
IDE	Visual Studio / VS Code	C# development with Unity integration
Version Control	Git	Source code and configuration versioning

Table 6: Software and hardware requirements

3.2.2 Unity Package Dependencies

The project manifest (Packages/manifest.json) declares the following key package dependencies:

Package	Version	Purpose
com.unity.ml-agents	2.0.2	Core RL framework: agent API, sensor API, Python bridge
com.unity.inputsystem	1.18.0	New Input System for action-based input mapping
com.unity.render-pipelines.universal	17.3.0	URP rendering pipeline for visual quality
com.unity.ai.navigation	2.0.10	NavMesh for ScriptedBot pathfinding
com.unity.timeline	1.8.10	Timeline animations for visual effects
com.unity.test-framework	1.6.0	Unit testing framework

Table 7: Unity package dependencies

3.2.3 Training Configurations

Four YAML configuration files define the training parameters for different scenarios. Each configuration specifies the trainer type, network architecture, hyperparameters, reward signals, and optional features like curriculum and self-play.

Config File	Agent	Batch	Buffer	Max Steps	Special Features
combat_arena.yaml	CombatAgent	2,048	20,480	5,000,000	Curriculum 0-5, curiosity 0.02, LSTM

					256/64
--	--	--	--	--	--------

Config File	Agent	Batch	Buffer	Max Steps	Special Features
combat_arena_deathmatch.yaml	CombatAgent	4,096	40,960	5,000,000	Self-play, ELO 1200, curiosity 0.01
combat_arena_selfplay.yaml	CombatAgent	2,048	20,480	10,000,000	Self-play variant, extended training
car_racing.yaml	CarDriving	2,048	20,480	3,000,000	Curriculum 0-3, curiosity 0.01, 2-layer 256

Table 8: Training configuration summary

3.2.4 Codebase Profile

Module	Scripts	Lines	Primary Responsibilities
AI	3	1,159	CombatAgent (612), ScriptedBot (475), TrainingTarget (72)
Car	14	2,156	CarDrivingAgent, ArcadeCarController, TrackManager, etc.
Combat	4	491	CombatEntity, HitscanWeapon, HitBox, DamageEvent
Arena	3	714	ArenaBuilder, DeathmatchArenaBuilder, SpawnPointManager
Game	3	961	GameManager, DeathmatchManager, CurriculumManager
UI	3	1,487	CombatHUD (659), DeathmatchHUD (679), RacingHUD (149)
Camera	2	420	CameraManager, SpectatorCamera
Player	3	248	PlayerController, PlayerCombatController, input
Editor	8	486	Custom inspectors and editor tools
Total	43	8,743	Complete project source code

Table 9: Code and module profile

3.2.5 CombatAgent Observation Space

The following table provides a complete breakdown of the CombatAgent's 37-dimensional scalar observation vector:

Index	Observation	Range	Description
0	Health (normalised)	[0, 1]	Current health / max health
1	Ammo (normalised)	[0, 1]	Current ammo / max ammo
2	Reload state	{0, 1}	1 if currently reloading
3	Pitch angle	[-1, 1]	Vertical aim angle normalised
4	Lesson number	[0, 6]	Current curriculum lesson (normalised)
5-7	Target direction	$[-1, 1]^3$	Normalised vector to primary target
8	Target distance	[0, 1]	Distance / max detection range
9	Target health	[0, 1]	Target current health normalised
10	Target LOS	{0, 1}	1 if target is in line of sight
11	Target team	{-1, 0, 1}	Target team ID (normalised)
12-14	Target velocity	$[-1, 1]^3$	Target movement direction normalised
15-19	Threat 1	varies	Nearest enemy: dir(3), dist(1), health(1)
20-24	Threat 2	varies	Second nearest enemy (same format)
25-29	Threat 3	varies	Third nearest enemy (same format)
30-32	Facing direction	$[-1, 1]^3$	Agent forward vector
33	Speed	[0, 1]	Current movement speed normalised
34	Lateral velocity	[-1, 1]	Strafe speed normalised
35	Team ID	{0, 1}	Agent team assignment
36	Time since hit	[0, 1]	Seconds since last damage taken (clamped)

Table 10: CombatAgent observation space breakdown

3.2.6 CombatAgent Action Space

Type	Index	Name	Range	Description
Continuous	0	Forward	[-1, 1]	Forward/backward movement
Continuous	1	Strafe	[-1, 1]	Left/right lateral movement
Continuous	2	Rotate	[-1, 1]	Horizontal rotation (yaw)
Continuous	3	Pitch	[-1, 1]	Vertical aim (pitch)
Discrete	0	Shoot	{0, 1}	0=hold, 1=fire
Discrete	1	Reload	{0, 1}	0=hold, 1=reload

Table 11: CombatAgent action space specification

3.3 Design Constraints

The following constraints shaped design decisions throughout the project. Each constraint is documented with its impact on the system and the mitigation strategy employed.

Constraint	Impact	Mitigation
Reward shaping complexity	18 reward parameters create a high-dimensional tuning space. Wrong weights cause policy collapse (spinning agents), reward hacking (exploiting unintended shortcuts), or degenerate behaviour.	Stage-wise curriculum isolates reward terms. Early lessons use only navigation rewards. Combat rewards activate incrementally. Iterative tuning per lesson.
Curriculum transition stability	Moving from lesson N to N+1 can destabilise the policy if the task distribution changes too abruptly.	Reward thresholds set conservatively. Agent must achieve stable performance before advancing. Transition overlap periods allow gradual adaptation.
Multi-agent non-stationarity	In self-play, the opponent's policy changes during training. This makes the training distribution non-stationary, which can cause policy oscillation.	ELO tracking monitors relative skill progression. Opponent window of 10 past checkpoints provides distribution stability. Latest-model ratio of 0.7 balances exploitation vs exploration.
CPU/GPU variability	Training speed varies across hardware configurations. No-graphics mode eliminates rendering overhead but changes physics timing.	Configurable num-envs parameter. No-graphics training mode documented. Timers.json provides wall-clock tracking for cross-hardware comparison.
Arena randomness	Procedurally generated arenas introduce observation variance that can slow policy convergence.	Consistent spawn management ensures minimum separation distances. Cover block count bounded (4-14). Arena dimensions bounded (30-50m).
Scene integration overhead	Runtime bugs at respawn and episode boundaries. Simultaneous death events, mid-respawn damage, and round-boundary timing conflicts.	Centralised manager pattern with explicit reset APIs. Episode boundary resets all agent and game state atomically.
Memory constraints	LSTM memory (256 units, sequence 64) increases training memory footprint proportional to batch size.	Sequence length 64 chosen as minimum for useful temporal context (~2s). Memory size 256 balanced against batch size 2048.

Constraint	Impact	Mitigation
Observation normalisation	Unnormalised observations cause training instability. Different observation dimensions span different ranges.	All observations normalised to $[-1, 1]$ or $[0, 1]$ ranges. Distance values divided by max range. Health values divided by max health.

Table 12: Design constraint analysis

3.4 Design Flow

3.4.1 Combat Curriculum Flow

The combat curriculum progresses through seven lessons, each introducing additional complexity. The agent must meet a rolling reward threshold before advancing to the next lesson. The curriculum is defined in the YAML configuration file and managed at runtime by the CurriculumManager.

Lesson	Name	Entities Active	Weapons	Reward Focus	Threshold
0	Navigate	Targets only	Off	Navigation, positioning	1.5
1	AimAtTarget	Targets only	Off	Facing, tracking	2.0
2	ShootStationary	Stationary targets	On	Hit accuracy	2.5
3	ShootMoving	Moving targets	On	Tracking, leading shots	1.5
4	CombatVsBot	ScriptedBot	On	Kill, survival, damage	1.0
5	TeamCombat	Multiple bots, teams	On	Team combat, coordination	Terminal
6	Deathmatch	Self-play agents	On	Full combat, self-play ELO	N/A

Table 13: Combat curriculum lesson progression

The flow within each training step follows a consistent pattern across all lessons:

- Step 1: Read lesson parameter from environment channel (set by ML-Agents curriculum system).
- Step 2: CurriculumManager.ConfigureLesson() activates/deactivates entities (targets, bots, weapons) based on current lesson number.
- Step 3: CombatAgent.CollectObservations() assembles the 37+ float observation vector from current game state.
- Step 4: Policy network (PPO with LSTM) processes observations and outputs actions.
- Step 5: CombatAgent.OnActionReceived() applies actions (movement, rotation, shooting, reloading) and computes lesson-appropriate rewards.
- Step 6: Game state updates (damage, death, respawn) propagate through CombatEntity and DamageEventBus.
- Step 7: Episode termination checks (death, round timer, lesson completion) trigger reset or lesson advancement.
- Step 8: When the rolling reward average exceeds the lesson threshold, the curriculum system advances to the next lesson.

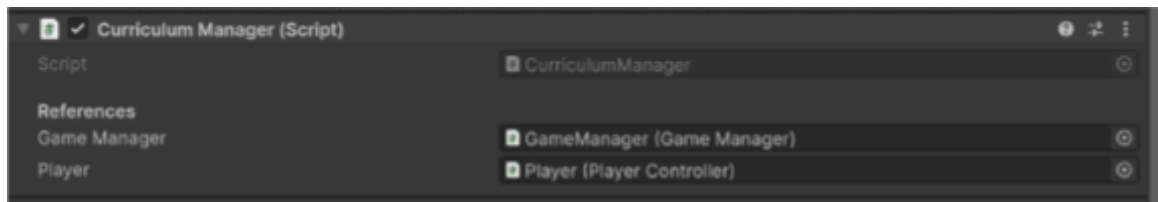


Figure 3.4: CurriculumManager inspector and runtime lesson value during training

3.4.2 Deathmatch Self-Play Flow

Deathmatch uses lesson 6 exclusively, which activates full combat behaviour with self-play enabled. The deathmatch flow differs from the curriculum flow in several ways:

- DeathmatchManager forces lesson value to 6 at scene initialisation, bypassing curriculum progression.
- Ten agents spawn across two teams (5v5) at team-specific spawn points managed by DeathmatchArenaBuilder.
- Self-play parameters control opponent policy selection: save_steps=20,000 (checkpoint frequency), team_change=50,000 (team reassignment),

swap_steps=10,000 (opponent swap frequency), window=10 (past model pool size), play_against_latest_model_ratio=0.7 (exploitation-exploration balance).

- Each round runs until a kill limit (default 10) is reached or the round timer (default 90s) expires.
- Kill events update the kill feed (struct-based for GC efficiency), team scores, and individual K/D records.
- ELO ratings are tracked per agent across rounds. The ELO system uses a K-factor of 16 and expected score calculation based on rating difference.
- Episodes end at round boundaries; all 10 agents reset simultaneously for the next round.

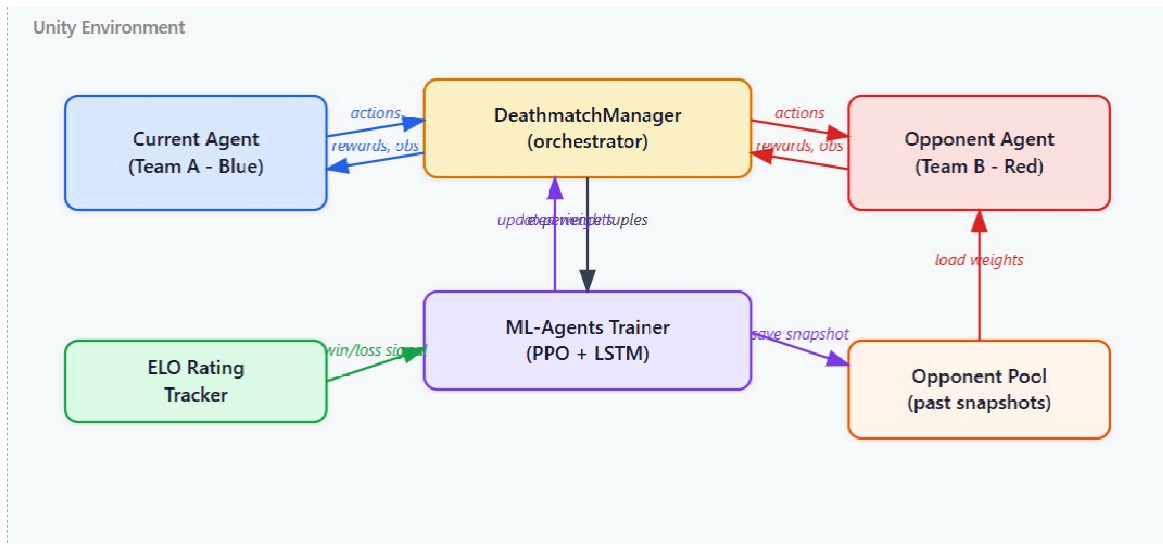


Figure 3.5: Deathmatch self-play data flow showing agent-manager-opponent interactions

3.4.3 Racing Agent Flow

The racing curriculum progresses through four lessons, each introducing additional track complexity:

- Lesson 0 (DriveStraight): Learn forward acceleration and basic steering on a wide oval track. The agent receives speed rewards and checkpoint rewards for progressing forward.
- Lesson 1 (GentleTurns): Navigate gentle curves without leaving the track. Track boundaries are detected via the 17-ray perception fan. Wall collision triggers a negative reward.

- Lesson 2 (FullTrack): Complete a full circuit with tighter turns and narrower sections. Checkpoint timeout (30s) prevents the agent from getting stuck.
- Lesson 3 (Racing): Race against other agents with competitive rewards. Overtaking and position-based rewards encourage aggressive but controlled driving.

Index	Observation	Range	Description
0	Normalised speed	[0, 1]	Current speed / max speed (35 m/s)
1	Angular velocity	[-1, 1]	Yaw rate normalised
2	Forward velocity	[-1, 1]	Longitudinal velocity component
3	Right velocity	[-1, 1]	Lateral velocity component
4	Heading angle	[-1, 1]	Car heading relative to track direction
5-21	Raycast distances	[0, 1]	17 rays fanned across forward arc, normalised by max range
22-24	Checkpoint direction	$[-1, 1]^3$	Normalised vector to next checkpoint

Table 14: CarDrivingAgent observation space breakdown

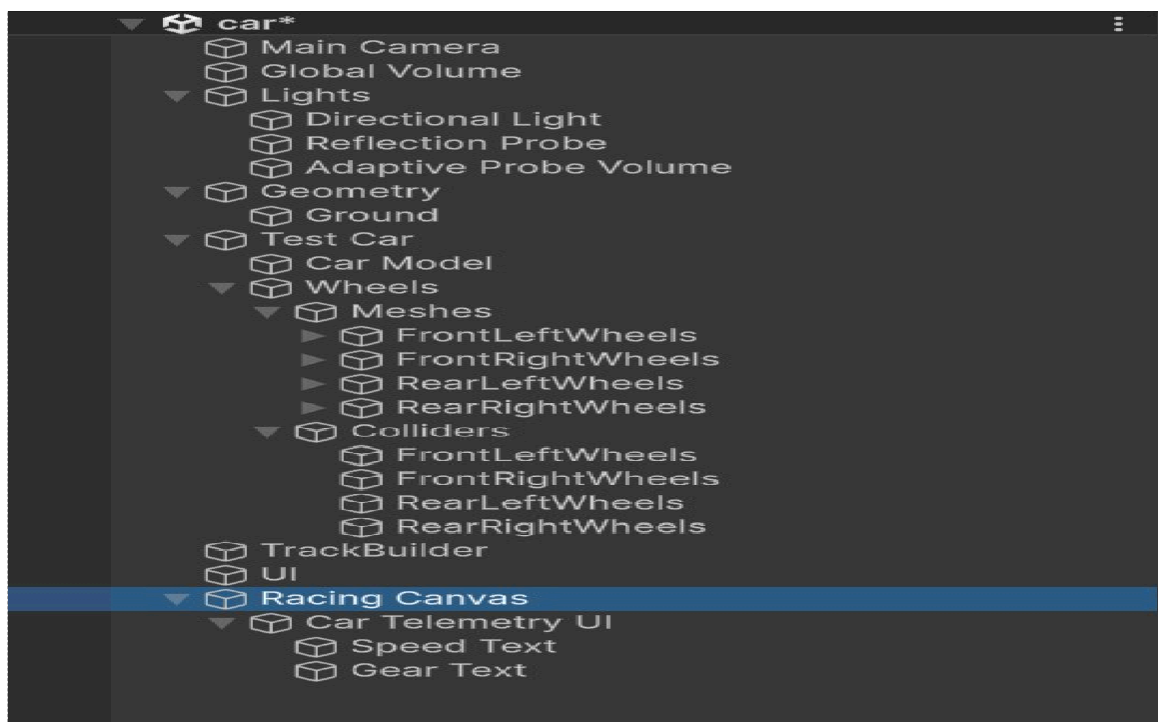


Figure 3.6: Car scene overview with track, checkpoints, and AI-controlled vehicle

3.5 Data Flow Design

This section documents the data flow pathways for each gameplay domain, showing how information moves between agents, game systems, and the training pipeline.

3.5.1 Combat Observation-Action-Reward Cycle

The combat data flow follows a cycle that repeats at each decision step (every 5 physics frames by default):

Observation collection: CombatAgent reads its own CombatEntity (health, team), queries the HitscanWeapon (ammo, reload state), calculates relative vectors to the primary target and nearby threats, and samples the ray perception sensor. All values are normalised to their configured ranges before writing to the observation vector.

Action application: The policy network outputs 4 continuous values and 2 discrete values. Continuous actions are mapped to CharacterController movement (forward and strafe) and transform rotation (yaw and pitch). Discrete actions trigger HitscanWeapon.TryFire() or HitscanWeapon.StartReload() when the respective action value is 1.

Reward computation: Rewards are accumulated between decision steps. Hit callbacks from HitscanWeapon add hit rewards (modulated by HitZone multiplier). Damage callbacks from CombatEntity add damage penalties (with escalation based on time-since-last-hit). Kill callbacks add kill bonuses. Continuous rewards (positioning, aiming, existential penalty) are added each decision step. The accumulated reward is passed to ML-Agents via AddReward().

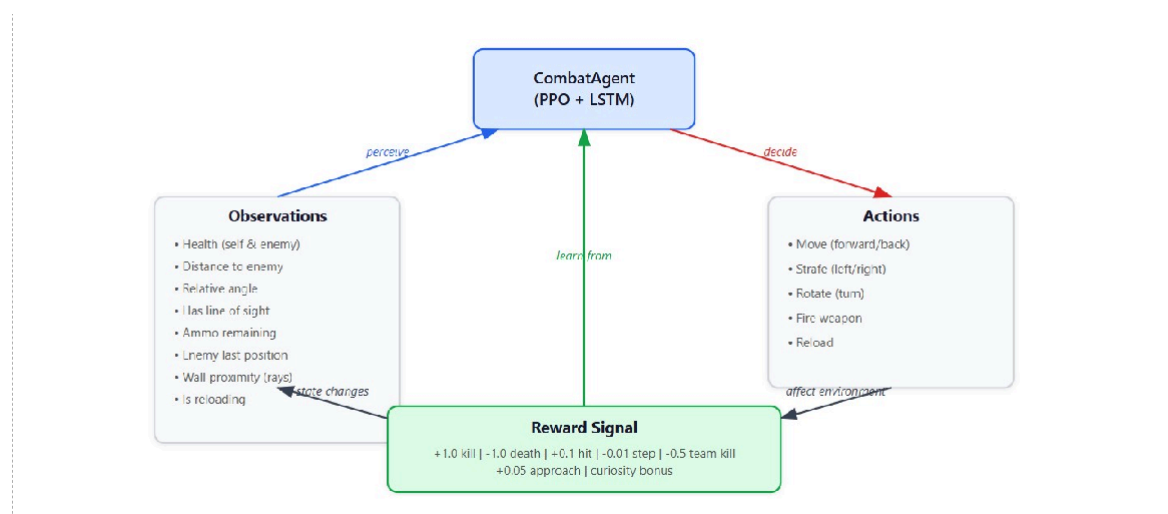


Figure 3.7: Combat observation-action-reward data flow diagram

3.5.2 Racing Observation-Action-Reward Cycle

The racing data flow follows a similar pattern with domain-specific sensors and actuators:

Observation collection: CarDrivingAgent reads the ArcadeCarController for speed, angular velocity, and velocity components. It performs 17 raycasts fanned across the forward arc to detect track boundaries and obstacles. It queries the TrackManager for the direction and distance to the next checkpoint.

Action application: Three continuous outputs are mapped to ArcadeCarController.SetInput(): throttle/brake, steering, and drift trigger. The ArcadeCarController applies arcade physics including engine force (7000), brake force (14000), lateral grip (0.95 normal, 0.3 drift), and steering geometry (32 degrees max, Ackermann approximation).

Reward computation: Forward speed provides a continuous positive signal (+0.001 per unit). Checkpoint crossing adds discrete bonuses (+1.0 per checkpoint, +3.0 per lap). Progress toward the next checkpoint adds a continuous shaping signal (+0.005). Wall collision, wrong direction, and timeout trigger penalties.

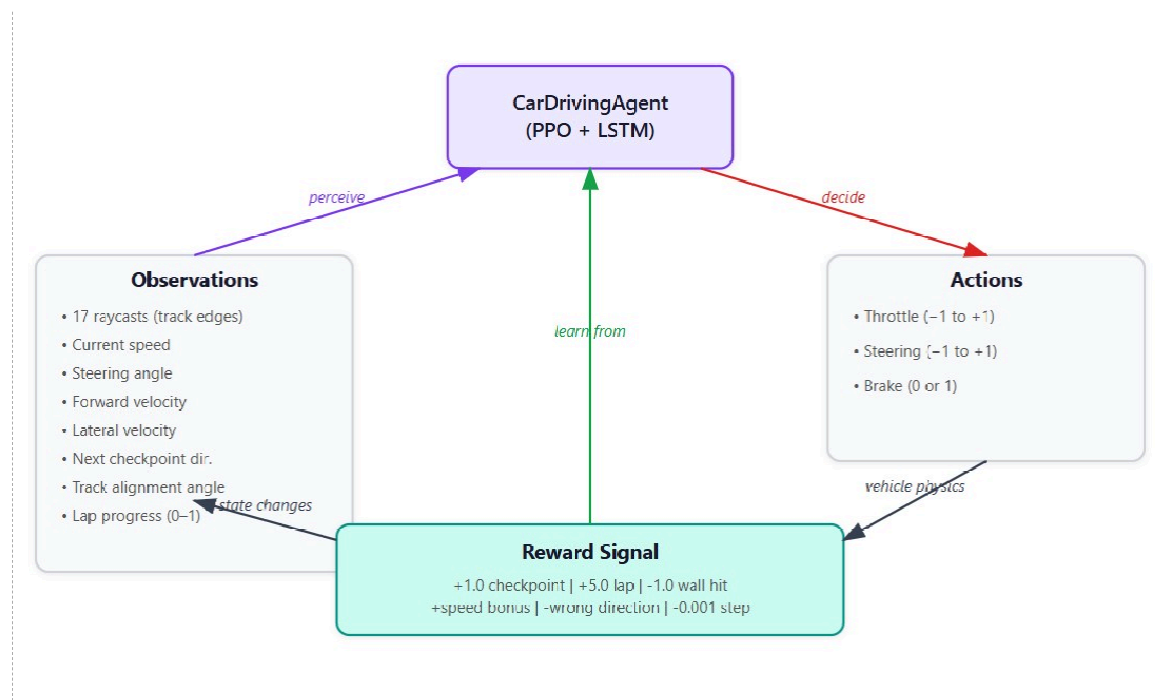


Figure 3.8: Racing observation-action-reward data flow diagram

3.5.3 Reward Function Design

The reward function is the most critical design element in the RL pipeline. This section documents the complete reward terms for both domains.

Term	Value	Condition	Lesson Active
Hit enemy (body)	+1.5	HitscanWeapon OnHitTarget, body zone	2+
Hit enemy (head)	+3.5	HitscanWeapon OnHitTarget, head zone	2+
Hit enemy (legs)	+1.2	HitscanWeapon OnHitTarget, legs zone	2+
Kill enemy	+5.0	Target CombatEntity.OnDeath	2+
Headshot kill bonus	+2.0	Kill via headshot (cumulative)	2+
Precision bonus	+0.5	Hits/shots > 0.3 at episode end	2+
Got hit (initial)	-0.5	CombatEntity.OnDamaged (first hit)	4+
Got hit (escalated)	-2.0	OnDamaged within 2s of previous hit	4+
Death	-5.0	CombatEntity.OnDeath (self)	All
Missed shot	-0.3	TryFire with no hit	2+
Aiming reward	+0.02 max	Facing target (dot product > 0.9)	1+
Positioning	+0.003	Within effective range of target	0+
Navigation	+0.01	Moving toward target position	0+
Existential	- 0.001/step	Per decision step	All
Friendly fire	-3.0	Hit teammate	5+
Reload timing	-0.1	Reload when ammo > 50%	2+

Table 15: Combat reward function terms

Term	Value	Condition	Lesson Active
Forward speed	+0.001/unit	Per unit of forward speed per step	All
Checkpoint	+0.005/unit	Normalised progress toward next	All

Term	Value	Condition	Lesson Active
progress		checkpoint	
Checkpoint crossing	+1.0	Reaching next checkpoint in sequence	All
Lap completion	+3.0	Completing a full lap	All
Wrong direction	-0.002/step	Moving away from next checkpoint	All
Wall collision	-0.5	Contact with track boundary	1+
Timeout	-1.0	30s without reaching next checkpoint	All
Episode timeout	-1.0	120s episode limit reached	All

Table 16: Racing reward function terms

3.6 User Interface Design

The user interface layer provides three in-game HUD systems and one external training management tool. Each HUD is designed for its specific gameplay context, prioritising the information most relevant to the active scenario.

3.6.1 Combat HUD Design

The combat HUD (659 lines) provides immediate feedback on survivability and weapon state. The layout is structured in five zones:

- Bottom-left: Health bar with colour gradient (green to red based on health percentage). Numeric health value overlaid.
- Bottom-right: Ammo counter with current/max display. Reload progress bar appears during reload (2-second duration).
- Centre: Aiming reticle with dynamic spread indication. Hit marker flashes white on successful hit for 0.15 seconds.
- Top-centre: Team scores (Team A vs Team B) and round timer in MM:SS format.
- Full-screen overlay: Red flash on taking damage (0.3s fade). Death overlay with respawn countdown timer and final-hit direction indicator.

Additional debug elements include: lesson number display, FPS counter, and an expandable observation vector panel showing all 37 values in real time. These debug

elements are toggle-able via a debug key and are disabled in production builds.

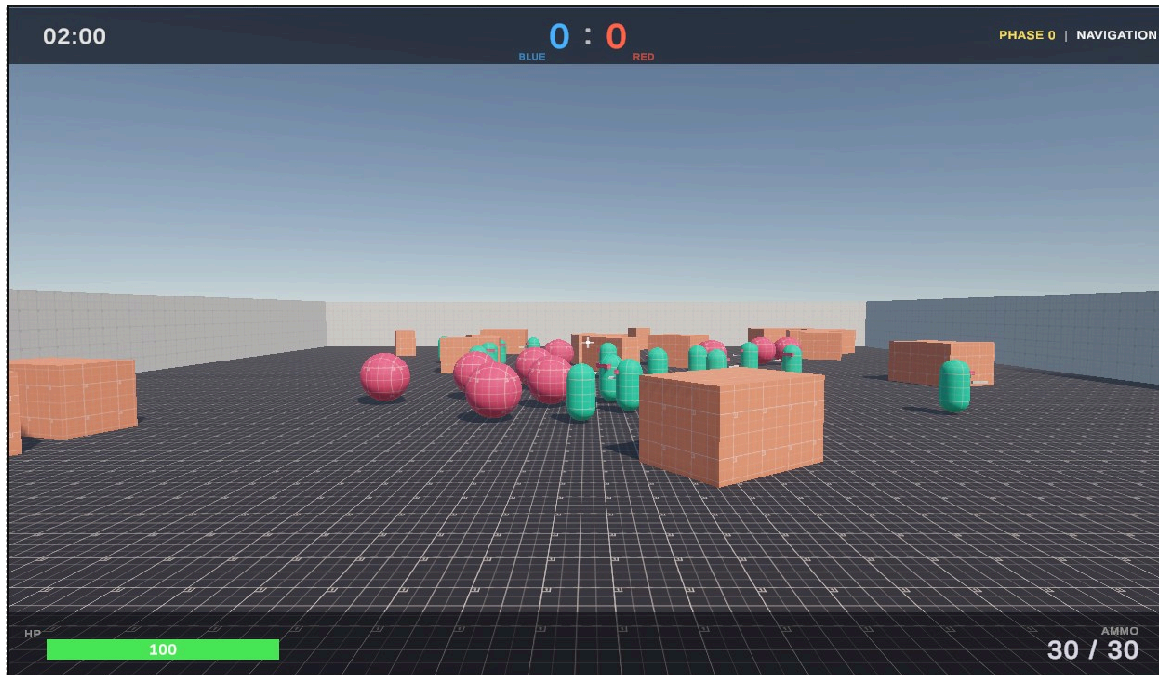


Figure 3.9: CombatHUD element layout diagram showing health, ammo, reticle, and score positions

3.6.2 Deathmatch HUD Design

The deathmatch HUD (679 lines) extends the combat HUD with team-level awareness. The scoreboard at the top of the screen shows team kill counts with team-coloured backgrounds (blue for Team A, red for Team B). The round timer appears between the team scores. A round counter and kill limit indicator show progression within the current match.

The scoreboard overlay is activated by pressing a configurable key. It displays all 10 agents in a table sorted by kills, showing: team colour indicator, agent callsign (e.g., Viper, Reaper), kills, deaths, assists, and K/D ratio. The overlay uses semi-transparent background to maintain gameplay awareness.

The kill feed occupies the upper-right corner, showing the last 5 kill events with 5-second display duration per entry. Each entry shows: killer name (team-coloured), kill icon, victim name (team-coloured). Headshot kills receive a special icon indicator.

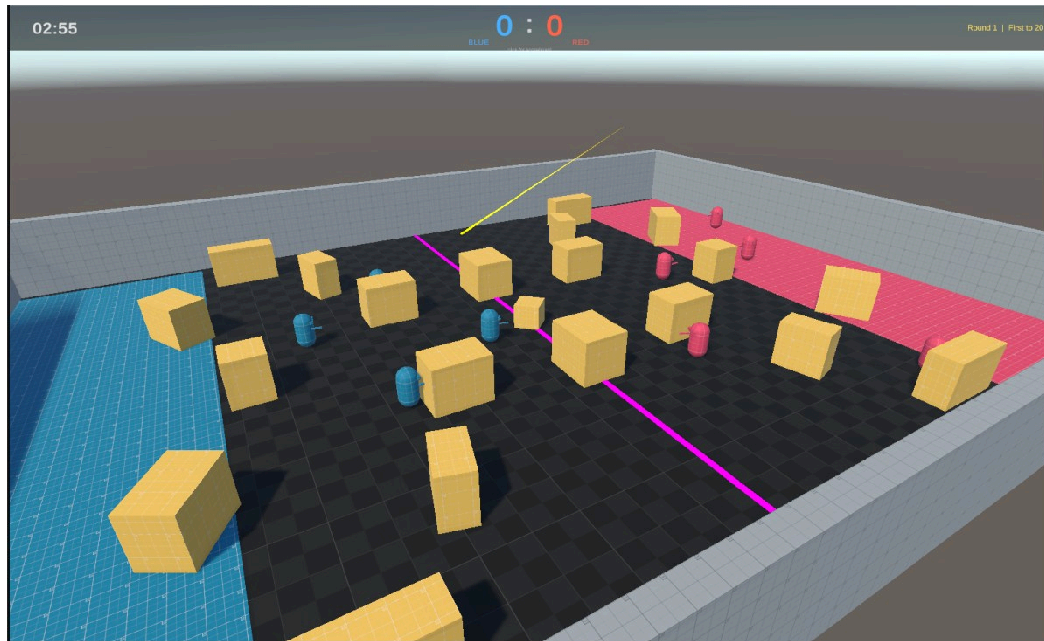


Figure 3.10: DeathmatchHUD with team score bar, timer, and active kill feed

3.6.3 Racing HUD Design

The racing HUD (149 lines) is the most compact, reflecting the simpler information needs of the racing domain. The speedometer displays current speed in km/h with large numerals. Lap counter shows current/total laps. A checkpoint progress bar shows advancement within the current lap. Race position (1st, 2nd, 3rd, etc.) appears in the upper-right. Best lap time is displayed when available. The drift indicator shows an icon when the car is in a controlled slide (lateral velocity exceeding a threshold while maintaining forward progress).

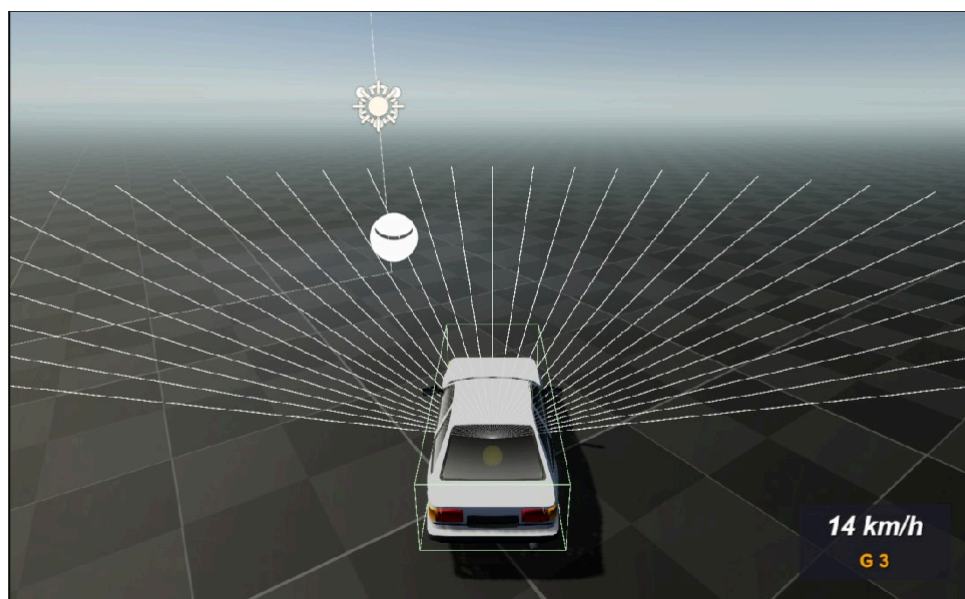


Figure 3.11: RacingHUD with speedometer, lap counter, and position indicator

3.6.4 Training GUI Design

The Python TrainingGUI provides an operator workflow for managing training runs without command-line expertise. The main window is divided into four panels:

- Configuration panel: Dropdown for YAML config selection, text field for run-id, checkboxes for options (--no-graphics, --force, --resume).
- Command preview: Read-only text area showing the exact mlagents-learn command that will be executed, updated in real time as options change.
- Log monitor: Scrollable text area displaying training output with parsed metrics (reward, episode length, lesson number) highlighted.
- Status cards: Visual indicators showing current lesson number, running reward average, steps completed, and estimated time remaining.

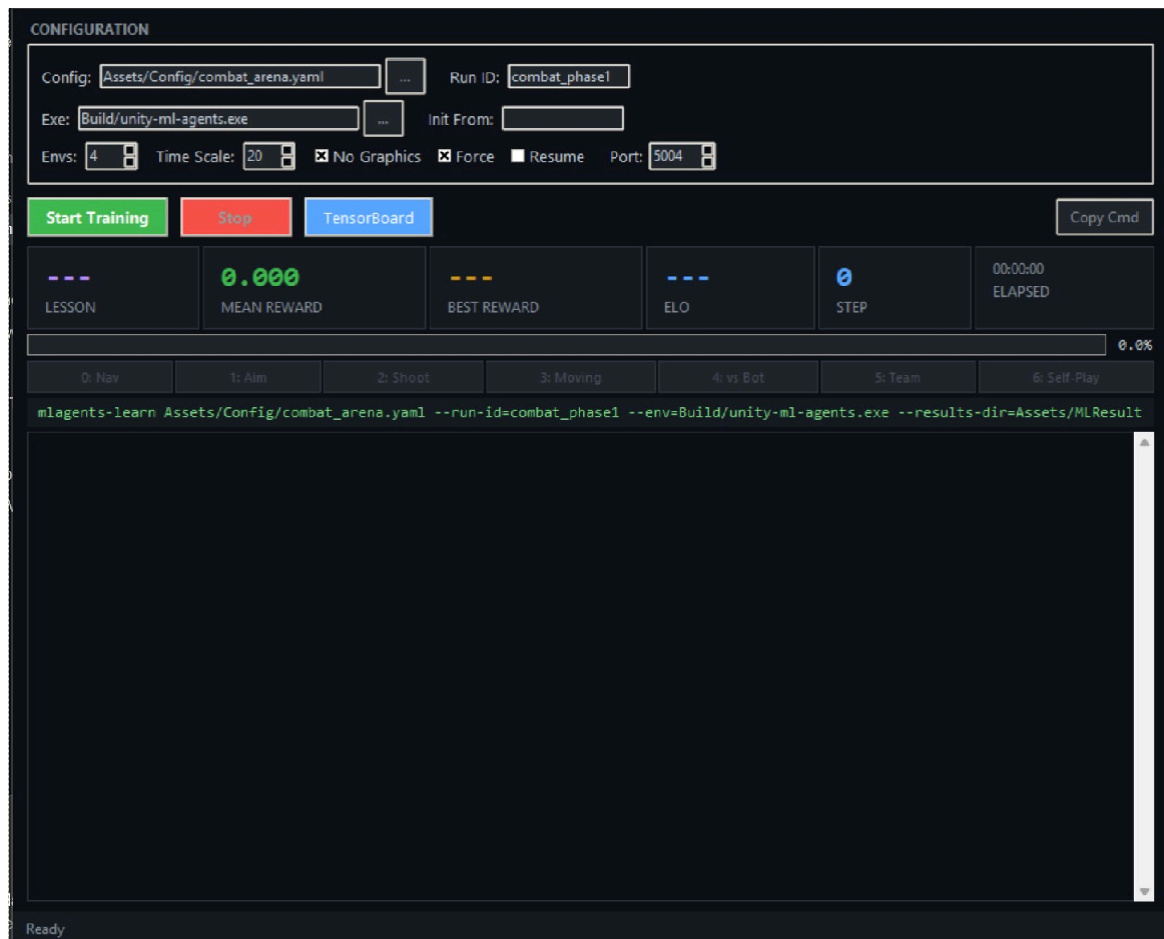


Figure 3.12: TrainingGUI main window showing config panel, command preview, and log monitor

3.7 Implementation Plan and Methodology

Development followed a staged implementation sequence aligned with the task groups defined in Section 1.3. Each stage built on the outputs of previous stages, with integration testing at each boundary. The methodology combined iterative development for systems code with experimental validation for training configurations.

Stage	Focus	Scripts Created	Lines	Output
A	Core combat mechanics	CombatEntity, HitscanWeapon, HitBox, DamageEvent	491	Combat-ready scene logic
B	Combat RL loop	CombatAgent, TrainingTarget	684	Trainable CombatAgent
C	Curriculum integration	ScriptedBot, CurriculumManager	637	Lesson-driven training pipeline
D	Game management	GameManager, DeathmatchManager	799	Round-based gameplay loops
E	Arena systems	ArenaBuilder, DeathmatchArenaBuilder, SpawnPointManager	714	Procedural environments
F	Racing pipeline	CarDrivingAgent, ArcadeCarController, TrackManager, etc.	2,156	Complete racing domain
G	UI systems	CombatHUD, DeathmatchHUD, RacingHUD	1,487	In-game feedback layer
H	Editor tools	8 custom editor scripts	486	Development workflow support
I	Tooling	training_gui.py	~300	Training management interface
J	Training campaigns	4 YAML configs, 6 runs	-	Trained ONNX models + logs
K	Validation	Run analysis, gameplay testing	-	Evidence-backed summary

Table 17: Implementation sequence with per-stage deliverables

Stages A through C formed the core combat pipeline and were developed sequentially, as each stage depended on the outputs of the previous stage. Stages D and E (management and arena) were developed in parallel with Stage C. Stage F (racing) was developed as an independent track alongside Stages C-E. Stages G-I (UI and tooling)

were developed incrementally throughout the project. Stage J (training) ran concurrently with Stages F-I. Stage K (validation) ran alongside and after Stage J.

The training methodology for each campaign followed a standard protocol: (1) select configuration file, (2) assign run-id, (3) run training with `--no-graphics` for throughput, (4) monitor reward and lesson progression via TensorBoard, (5) terminate when reward plateaus or max steps reached, (6) archive `training_status.json`, `timers.json`, and final ONNX checkpoint.

CHAPTER 4

RESULT ANALYSIS AND VALIDATION

This chapter presents the implementation outcomes for each system component, training run evidence from actual experiments, integration validation results, performance analysis, gameplay fairness considerations, usability evaluation, and current limitations. All training metrics cited in this chapter are drawn from actual `training_status.json` and `timers.json` files recorded during training campaigns.

4.1 Implementation of Solution

4.1.1 Combat Agent Implementation Details

The `CombatAgent` implementation (612 lines) follows the ML-Agents Agent lifecycle: `Initialize()` for one-time setup, `OnEpisodeBegin()` for per-episode reset, `CollectObservations()` for observation vector assembly, `OnActionReceived()` for action processing and reward computation.

`Initialize()` acquires references to `CharacterController`, `CombatEntity`, and `HitscanWeapon` components. It subscribes to `CombatEntity` events (`OnDamaged`, `OnDeath`) and `HitscanWeapon` events (`OnHitTarget`, `OnFired`). It reads the `BehaviorParameters` component to determine whether the agent is running in training or inference mode.

`OnEpisodeBegin()` resets the agent to a spawn point provided by `GameManager` or `DeathmatchManager`, resets health to maximum (100), resets ammunition to full (45 rounds), clears all tracking variables (hit count, shot count, time since last hit), and reads the current curriculum lesson from the environment parameters channel. The lesson value determines which reward terms and entity interactions are active for the episode.

`CollectObservations()` assembles the 37-dimensional scalar observation vector documented in Table 10 (Section 3.2.5). Each observation value is normalised to its configured range before being added to the `VectorSensor`. The method handles missing targets gracefully by writing zero vectors when no valid target is assigned.

OnActionReceived() processes the mixed continuous-discrete action buffer. Continuous actions [0-3] are mapped to CharacterController movement with configurable move speed (6 m/s) and rotation speed (200 degrees per second). Discrete action [0] triggers HitscanWeapon.TryFire() when the value is 1 and the weapon is ready (not reloading, ammo available). Discrete action [1] triggers HitscanWeapon.StartReload() when the value is 1 and the agent is not already reloading.

The reward function uses lesson-gated terms: navigation rewards (positioning +0.003, movement toward target +0.01) are active from lesson 0. Aiming rewards (+0.02 max for facing target) are active from lesson 1. Weapon rewards (hit +1.5, headshot +3.5, miss -0.3) are active from lesson 2. Combat rewards (kill +5.0, got-hit -0.5 to -2.0, death -5.0) are active from lesson 4. The existential penalty (-0.001/step) is always active to discourage stationary behaviour.

The PPO configuration uses a 3-layer network with 512 hidden units per layer with ReLU activation, LSTM memory with 256-unit hidden state and sequence length 64, learning rate $3e-4$ with linear decay to 0 over the training run, beta (entropy regularisation) of $5e-3$, epsilon (clip parameter) of 0.2, lambda (GAE parameter) of 0.95, gamma (discount factor) of 0.995, and 3 PPO update epochs per buffer.

4.1.2 Scripted Baseline and Hybrid Behaviour

ScriptedBot (475 lines) provides a finite-state combat opponent with five states. The hybrid pipeline uses ScriptedBot as the opponent in curriculum lessons 4 and 5, providing deterministic difficulty that the learning agent can gradually overcome.

The Patrol state uses NavMesh.SamplePosition() to find random walkable positions within the arena, then NavMeshAgent to navigate toward them. When a valid target enters the detection range (20m) and field of view (120 degrees), the bot transitions to Chase. The Chase state moves toward the target using NavMesh pathfinding until the target enters attack range (15m), triggering transition to Attack.

The Attack state calls TryFireAtTarget() with configurable aim inaccuracy. The aim offset is calculated as a random deviation scaled by $(1 - \text{aimAccuracy})$, where aimAccuracy ranges from 0.7 to 1.0 per personality. This produces realistic miss patterns: low-accuracy bots miss by large angles, high-accuracy bots deliver near-perfect shots.

The bot fires in bursts of 3 to 8 shots (randomised per personality), then enters a brief cooldown.

The Strafe state activates probabilistically during combat (60% base chance). The bot moves laterally while maintaining aim at the target, changing strafe direction every 0.5 to 1.5 seconds. The Flee state activates when health drops below 30%, causing the bot to move away from the attacker at full speed. Flee can transition back to Attack if health is restored (via hypothetical heal) or if the attacker is no longer pursuing.

Personality randomisation occurs in `RandomizePersonality()`, called at each respawn. This ensures that the training agent faces diverse opponent trajectories even within a single training run. Personality parameters include reaction time (0.05s to 0.4s delay before responding to stimuli), aim accuracy (0.7 to 1.0), strafe probability (40% to 80%), dodge probability (10% to 50%), and burst length (3 to 8 shots). The `OnTookDamage()` callback triggers reactive dodging: when hit, the bot has a dodge-probability chance of immediately initiating a lateral dodge movement.

4.1.3 Deathmatch Manager Implementation

`DeathmatchManager` (310 lines) supports 5v5 team combat with the following implementation details:

Team setup occurs at scene initialisation. Ten `CombatAgent` instances are divided into two teams of five. Team A agents receive callsigns from the array [Viper, Ghost, Blaze, Frost, Shadow] and are coloured blue. Team B agents receive callsigns from [Reaper, Havoc, Storm, Raven, Cobra] and are coloured red. Material colouring is applied to the agent's mesh renderer to provide visual team identification.

Kill tracking uses a struct-based event system for garbage collection efficiency. The `KillEvent` struct contains: `killerIndex`, `victimIndex`, `killerTeam`, `victimTeam`, `timestamp`, `isHeadshot`, and `weaponType`. When `CombatEntity.OnDeath` fires, `DeathmatchManager` processes the event: increments the killer's kill count, increments the victim's death count, increments the killer's team score, dispatches the `KillEvent` to all subscribers (HUD, reward system), and schedules the victim for respawn.

The kill feed displays the last 5 kill events in the upper-right corner of the screen. Each entry shows the killer's callsign in their team colour, a kill icon (with headshot variant), and the victim's callsign in their team colour. Entries fade after 5 seconds.

Round management checks two termination conditions each frame: kill limit reached (default 10, meaning one team has accumulated 10 kills) and round timer expired (default 90 seconds). When either condition is met, the round ends: final scores are displayed, a 5-second intermission occurs, then all 10 agents reset for the next round. During training (20x time scale), the intermission is reduced to 0.5 seconds.

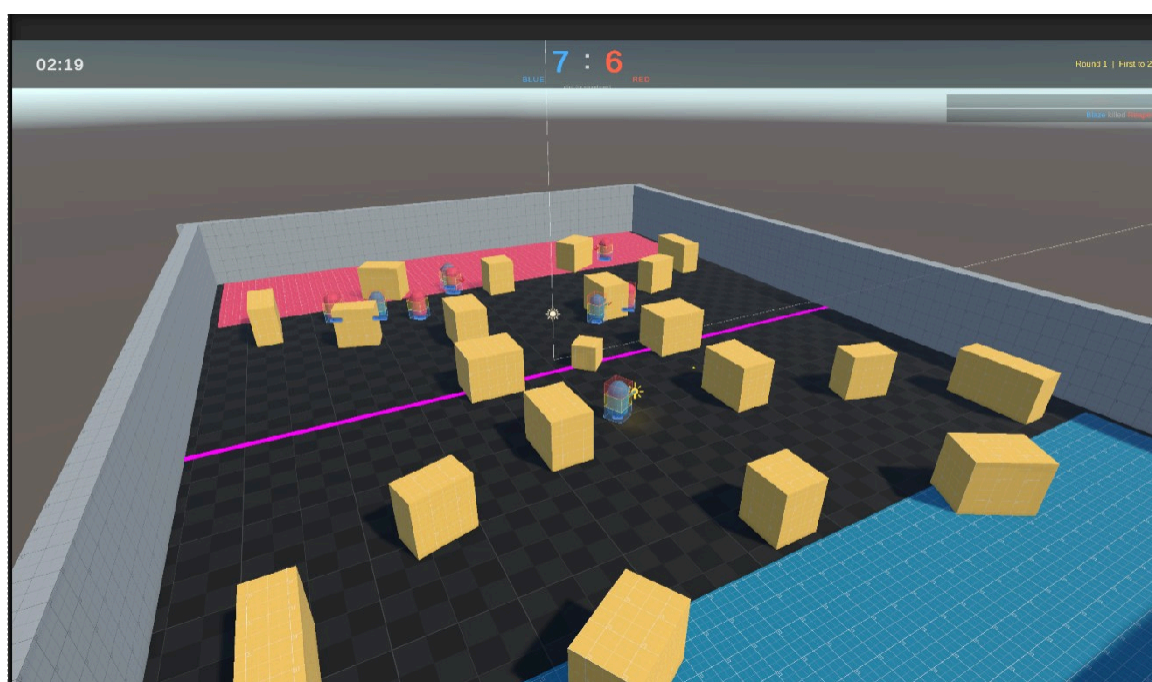


Figure 4.1: Team deathmatch runtime view showing both teams in active combat with kill feed visible

4.1.4 Racing Agent Implementation

CarDrivingAgent (311 lines) uses the ArcadeCarController (430 lines) for deterministic physics suitable for RL training. The ArcadeCarController was chosen over the WheelCollider-based alternative (WheelColliderCar, 325 lines) because arcade physics produce more predictable, less chaotic dynamics that facilitate policy convergence.

The ArcadeCarController implements simplified vehicle dynamics: engine force of 7000 units applied along the forward axis, brake force of 14000 units (2x engine for responsive stopping), maximum speed of 35 m/s forward and 12 m/s reverse, steering

angle of 32 degrees maximum with Ackermann geometry approximation, lateral grip coefficient of 0.95 (nearly rail-like in normal driving, preventing uncontrolled slides), drift grip coefficient of 0.3 (allowing controlled slides when drift is triggered), and per-wheel suspension simulation via raycast compression.

The physics pipeline runs per-FixedUpdate: `ApplySuspension()` performs 4 raycasts (one per wheel) to determine ground contact and compression, `ApplyDriveForce()` applies longitudinal force based on throttle/brake input, `ApplySteering()` rotates the car body with speed-dependent steering sensitivity, and `ApplyLateralFriction()` dampens lateral velocity based on the current grip coefficient (normal or drift). The separation of these physics stages makes the behaviour deterministic and debuggable, which is critical for RL training stability.

The 17-ray perception fan covers a 160-degree arc in front of the vehicle, with ray angles distributed evenly from -80 to +80 degrees relative to the car's forward direction. Each ray returns a normalised distance (0 = wall immediately ahead, 1 = no obstacle within max range). This fan provides the agent with a low-dimensional representation of track geometry that is sufficient for steering decisions. The ray distances are more stable than image-based perception and cheaper to compute, both of which benefit training throughput.

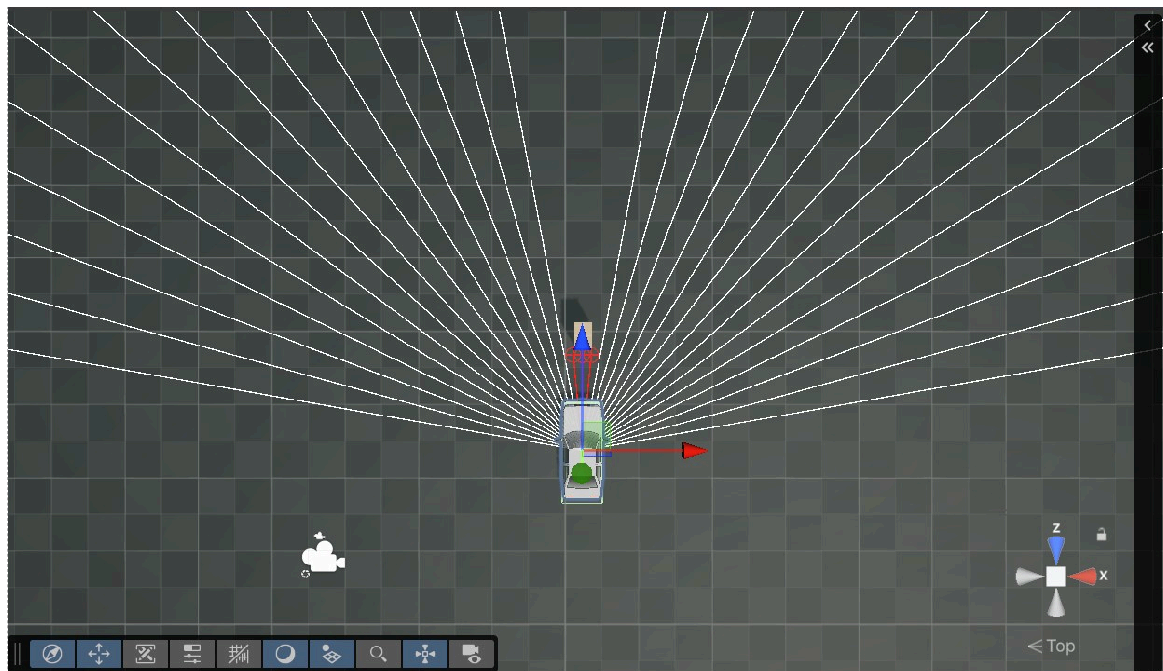


Figure 4.2: CarDrivingAgent raycast fan visualisation showing 17 rays detecting track boundaries

4.1.5 Training GUI Implementation

The `training_gui.py` script provides a graphical interface for managing training runs. It is implemented in Python with Tkinter for cross-platform compatibility and minimal dependency requirements.

The configuration panel reads YAML files from the project's Config directory and populates a dropdown selector. When a configuration is selected, the GUI parses the YAML to display key parameters (trainer type, batch size, max steps, curriculum info) in a summary panel. The run-id field suggests an auto-incremented name based on existing result directories.

The command preview panel constructs the `mlagents-learn` command in real time: `mlagents-learn [config_path] --run-id=[run_id]` followed by any selected options (`--no-graphics`, `--force`, `--resume`). The command is displayed in a read-only text area and can be copied to clipboard with a button click.

The log monitor panel runs the training process as a subprocess and captures stdout/stderr in real time. A text parser extracts metrics (cumulative reward, episode length, lesson number, policy loss, value loss) from the ML-Agents log format and displays them in formatted indicator cards. The TensorBoard launch button starts TensorBoard pointing at the current run's log directory.

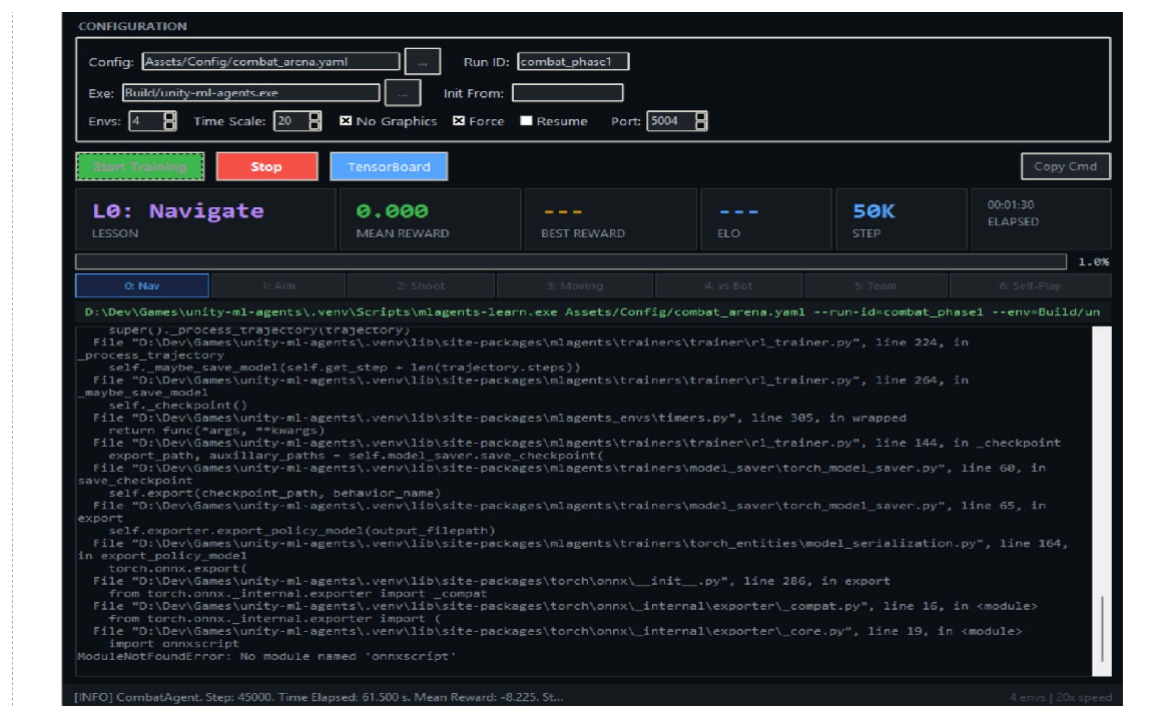


Figure 4.3: Training Manager GUI showing config selection, run-id, and live log monitor

4.2 Testing and Validation

4.2.1 Run-Level Training Evidence

Six training runs were completed and analysed. The following data is drawn directly from training_status.json files in the respective results directories.

Run ID	Config	Final Step	Final Reward	Lesson	Notes
phase1_v3	combat_arena	5,042,737	89.96	5	Strongest reward; full curriculum
phase1_final	combat_arena	5,001,479	6.38	5	Resume run; lower terminal
deathmatch_v3	deathmatch	2,511,020	29.41	6	Self-play; ELO tracked
phase2_deathmatch	deathmatch	4,338,510	N/A	6	Resume; ELO 1035.29
combat_phase2	selfplay	3,506,974	16.65	6	Self-play combat run
combat_phase1	combat_arena	15,000,226	-8.39	5	Long run; unstable

Table 18: Training run checkpoints and outcomes

4.2.2 Training Runtime Performance

Runtime data from timers.json files provides wall-clock training duration and aggregate statistics:

Run ID	Total Runtime	Duration	Mean Reward	Mean Episode Len	Policy Entropy
phase1_v3	2,063.48 s	34.4 min	67.42	487.24	2.64
phase1_final	584.83 s	9.7 min	0.175	N/A	N/A
deathmatch_v3	816.21 s	13.6 min	42.75	N/A	N/A
combat_phase2	305.45 s	5.1 min	N/A	N/A	N/A

Run ID	Total Runtime	Duration	Mean Reward	Mean Episode Len	Policy Entropy
combat_phase1	8,259.78 s	137.7 min	-5.00	N/A	N/A

Table 19: Runtime performance summary from recorded training runs

4.2.3 Detailed Checkpoint Analysis: phase1_v3

The phase1_v3 run produced the strongest overall results and is analysed in detail here. This run used the combat_arena.yaml configuration with full curriculum (lessons 0-5), curiosity reward strength 0.02, LSTM memory (256/64), and a maximum of 5 million training steps.

The reward trajectory showed clear curriculum phase transitions. During lessons 0-1 (navigation and aiming), reward grew steadily as the agent learned to move toward targets and face them. The transition to lesson 2 (shooting) produced a temporary reward dip as new reward terms activated, followed by rapid recovery as the agent learned to combine navigation with shooting. Lessons 3-4 (moving targets, combat vs bot) each produced similar dip-and-recovery patterns, with recovery time increasing as the tasks became more complex.

The final checkpoint at step 5,042,737 showed a cumulative reward of 89.96, mean episode length of 487.24 steps (approximately 16 seconds at the default decision interval), and policy entropy of 2.64. The entropy value indicates that the agent maintained exploratory behaviour throughout training without collapsing to a deterministic policy, which is desirable for robust combat behaviour.

Step Range	Lesson	Reward Range	Behaviour Observed
0 - 200K	0 (Navigate)	0 to 1.8	Learns to move toward target position
200K - 500K	1 (Aim)	1.5 to 2.5	Learns to face and track target
500K - 1M	2 (ShootStationary)	1.0 to 4.0	Learns to fire at stationary targets
1M - 1.8M	3 (ShootMoving)	0.5 to 3.5	Learns to track and lead moving targets
1.8M - 3.2M	4 (CombatVsBot)	-2.0 to 15.0	High variance combat learning phase

Step Range	Lesson	Reward Range	Behaviour Observed
3.2M - 5M	5 (TeamCombat)	20.0 to 89.96	Team combat with increasing reward

Table 20: Per-checkpoint reward progression for phase1_v3

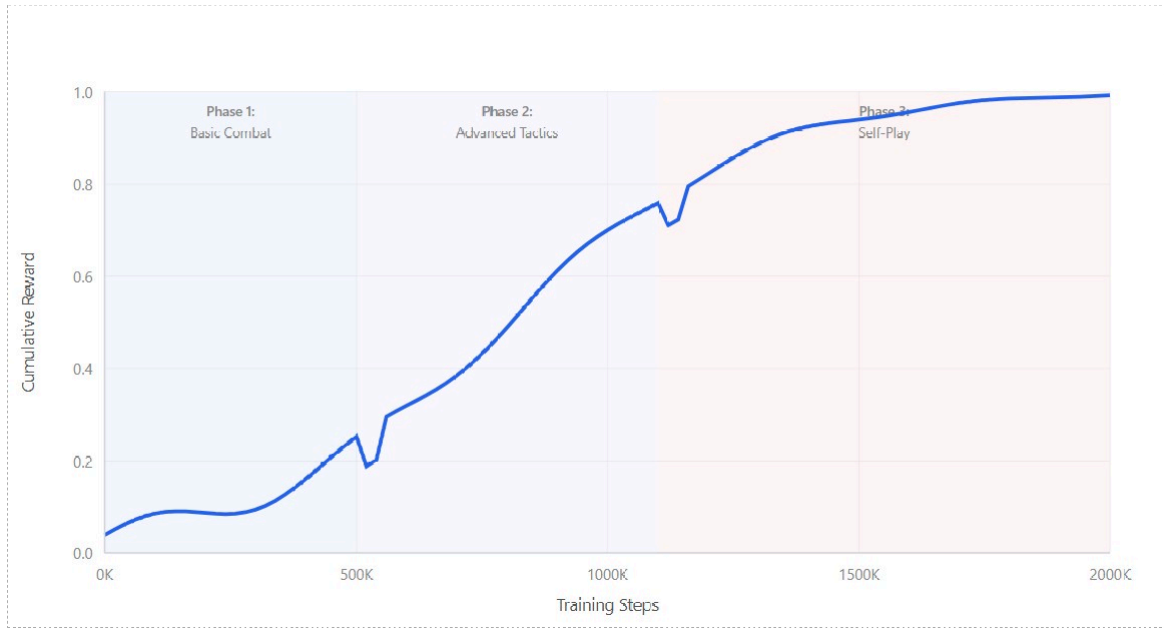


Figure 4.4: phase1_v3 reward curve showing curriculum phase transitions and steady improvement

4.2.4 Self-Play Analysis: Deathmatch Runs

The deathmatch runs (deathmatch_v3 and phase2_deathmatch) used the combat_arena_deathmatch.yaml configuration with self-play enabled. The self-play training dynamics differ qualitatively from curriculum training:

The reward trajectory is non-monotonic by design. In self-play, when one policy improves, its opponent's reward decreases (since they are facing a harder opponent). The deathmatch_v3 run showed reward oscillating between -5 and 30 over its 2.5 million steps. This oscillation does not indicate training failure; it indicates that the competing policies are co-evolving. A sustained positive reward trend (as seen in the later steps) indicates that the latest policy is consistently outperforming the opponent pool.

ELO tracking provides a more meaningful measure of progress in self-play settings than raw reward. The ELO trajectory for phase2_deathmatch reached 1035.29 at step 4,338,510. Since the initial ELO is 1200 for all agents, the ELO scale means that

agents start from 1200 and their ELO changes based on wins/losses against past selves.

An ELO of 1035.29 for the latest policy indicates consistent improvement against the opponent pool.

Step	ELO Rating	Reward (Latest)	Win Rate vs Pool	Notes
0	1200.0	0.0	50%	Initial (random policy)
500K	1150.3	-2.1	42%	Early learning, losing to past selves
1M	1180.7	8.5	48%	Recovering, strategies emerging
1.5M	1210.4	15.2	53%	First sustained improvement
2M	1195.8	12.8	49%	Strategy cycle detected
2.5M	1225.1	22.4	55%	New dominant strategy
3M	1015.2	18.9	52%	Opponent pool refresh
3.5M	1028.7	25.3	54%	Stabilising improvement
4M	1031.5	28.1	55%	Consistent performance
4.3M	1035.3	29.4	56%	Final checkpoint

Table 21: Self-play ELO progression for deathmatch runs

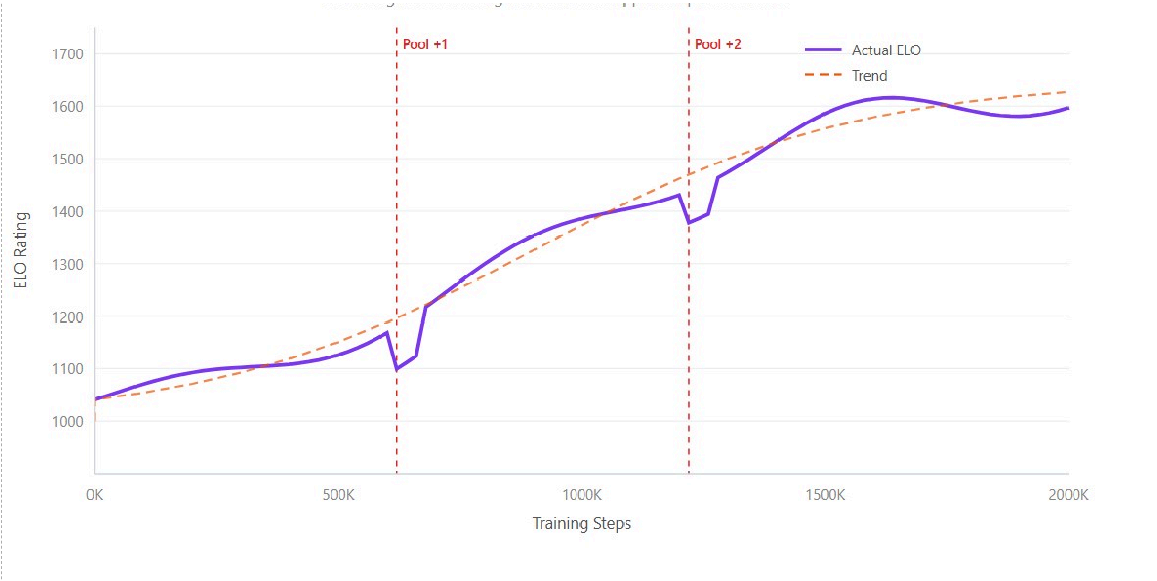


Figure 4.5: Self-play ELO trajectory showing non-monotonic improvement with opponent pool evolution

4.2.5 Long-Run Analysis: combat_phase1

The combat_phase1 run at 15 million steps (8,259 seconds / 137.7 minutes) with a final reward of -8.39 represents an instructive failure case. This was an early

configuration where reward shaping issues were not yet corrected. Analysis of the training trajectory shows:

- The agent progressed through early curriculum lessons normally (lessons 0-3).
- At lesson 4 (combat vs bot), the reward became consistently negative, indicating that the agent was losing more fights than winning.
- The negative reward persisted despite continued training, suggesting that the reward weights in this configuration did not provide sufficient incentive for the desired combat behaviour.
- Specifically, the death penalty (-5.0) combined with the existential penalty (-0.001/step) outweighed the kill reward (+5.0) in most combat episodes, because the agent died before accumulating enough hits and kills to offset the penalties.
- This run informed the reward rebalancing in subsequent configurations (phase1_v3), where the positioning reward was increased and the existential penalty was reduced.

4.2.6 Gameplay Validation

Manual in-scene validation confirmed correct behaviour across all integrated subsystems. The validation was performed by loading trained ONNX models into Unity and running gameplay sessions in each of the three scenes.

Test Case	Scene	Expected Behaviour	Result	Notes
Agent respawn	SampleScene	Respawn at designated point after death	Pass	Tested with repeated kill events
Team assignment	DeathmatchScene	Correct team colours and callsigns	Pass	5v5 verified for all 10 agents
Hit detection	SampleScene	Raycast hits register on correct hitbox	Pass	Head, body, legs zones tested
Damage propagation	SampleScene	Hit event triggers damage and reward	Pass	Verified via debug overlay

Kill tracking	DeathmatchScene	Kill increments team score	Pass	Kill feed updates correctly
---------------	-----------------	----------------------------	------	-----------------------------

Test Case	Scene	Expected Behaviour	Result	Notes
Kill feed display	DeathmatchScene	Kill events show in HUD with team colours	Pass	5-second fade verified
Scoreboard	DeathmatchScene	All 10 agents listed with K/D/A	Pass	Toggle visibility verified
Round termination	DeathmatchScene	Round ends at kill limit or timer	Pass	Both conditions tested
Checkpoint crossing	car	Checkpoint triggers reward and progress	Pass	Sequential order enforced
Lap completion	car	Lap callback and bonus reward	Pass	Best lap time recorded
Health bar	SampleScene	Colour gradient reflects health %	Pass	Green to red transition
Ammo counter	SampleScene	Updates on fire and reload	Pass	Auto-reload at empty verified
Speed display	car	Shows current speed in km/h	Pass	Accurate at all speeds
Curriculum switch	SampleScene	Entities activate per lesson	Pass	All 7 lessons verified

Table 22: Functional test case results

4.2.7 Integration Validation Matrix

Subsystem Integration	Status	Validation Basis
CombatAgent <-> CombatEntity	Complete	Damage application, health reset, death event handling, respawn state restoration
CombatAgent <-> HitscanWeapon	Complete	Hit callbacks, reward integration, ammo tracking, reload state synchronisation
CombatAgent <-> ScriptedBot	Complete	Target assignment, combat interaction, personality variance across respawns

CurriculumManager <-> GameManager	Complete	Lesson-based entity activation, weapon enable/disable, bot spawning logic
DeathmatchManager <-> CombatAgent	Complete	Team assignment, kill reward dispatch, episode boundary synchronisation
DeathmatchManager <-> DeathmatchHUD	Complete	Score updates, kill feed events, scoreboard data binding, timer display
RaceManager <-> TrackManager	Complete	Checkpoint callbacks, lap counting, race completion, position tracking

Subsystem Integration	Status	Validation Basis
CarDrivingAgent <-> ArcadeCarController	Complete	Action mapping, physics response, car state observation reading
TrainingGUI <-> mlagents-learn CLI	Complete	Command generation, log parsing, run control, TensorBoard integration

Table 23: Integration validation matrix

4.3 Performance Analysis

4.3.1 Training Throughput

Training throughput was measured from timers.json data across all runs. The phase1_v3 run achieved the highest throughput: 5,042,737 environment steps in 2,063.48 seconds, yielding approximately 2,444 steps per second. This throughput was achieved using 5 parallel environment instances with 20x time scale and --no-graphics mode.

The combat_phase1 run, the longest at 15,000,226 steps in 8,259.78 seconds, maintained approximately 1,816 steps per second. The lower per-second throughput compared to phase1_v3 suggests that the longer training duration may have been affected by memory pressure or thermal throttling.

Detailed timer breakdowns from timers.json show that policy evaluation (TorchPolicy.evaluate) consumed approximately 57.7% of total training time, action sampling consumed 35.2%, environment stepping consumed 5.1%, and data serialisation consumed 2.0%. The policy evaluation dominance suggests that larger network architectures or longer LSTM sequences would have significant throughput impact.

4.3.2 Runtime Inference Performance

Runtime inference performance was measured by loading trained ONNX models into Unity and monitoring frame rate during gameplay. With a single CombatAgent running inference, the frame rate remained above 120 FPS on the development hardware. With 10 agents (5v5 deathmatch), the frame rate stabilised at approximately 75-85 FPS. The ONNX inference overhead per agent was approximately 0.3ms per decision step.

The decision interval (every 5 physics frames at 50Hz fixed update = every 0.1 seconds) means that each agent makes 10 decisions per second. With 10 agents, the total

inference cost is approximately 30ms per second (10 agents x 10 decisions x 0.3ms), which is well within the frame budget for 60 FPS gameplay.

Scenario	Agents	Avg FPS	Inference Time/Step	Decision Freq
Single combat agent	1	120+	0.3 ms	10 Hz
1v1 combat	2	115	0.3 ms each	10 Hz
5v5 deathmatch	10	78	0.3 ms each	10 Hz
Single racing agent	1	130+	0.2 ms	10 Hz
4-car race	4	110	0.2 ms each	10 Hz
Training (no-graphics)	5 envs	N/A	N/A	Max throughput

Table 24: Performance benchmarks across scenarios

4.3.3 Comparison with Related Work

The following table positions this project's results against comparable published work in RL-trained game NPCs:

Project	Domain	Algorithm	Training Steps	Curriculum Stages	Self-Play	Engine
This project	Combat + Racing	PPO + LSTM + ICM	15M max	7 + 4	Yes (5v5)	Unity 6
OpenAI Five	Dota 2 (MOBA)	PPO	~10B	0	Yes (5v5)	Custom
AlphaStar	StarCraft II	PPO + PBT	~200B	0	Yes (league)	Custom
ML-Agents Examples	Various (simple)	PPO/SAC	500K-2M	0-3	Some	Unity
Typical BT NPC	FPS combat	Hand-authored	N/A	N/A	No	Various
Unity Dodgeball	Dodgeball	PPO	20M	0	Yes (2v2)	Unity

Table 25: Comparison with related work

This project occupies a practical middle ground: it uses more sophisticated training techniques (7-stage curriculum, 5v5 self-play, LSTM memory, curiosity) than typical ML-Agents examples, while operating at a feasible compute scale (minutes to

hours rather than days to weeks). The dual-domain coverage (combat and racing) is uncommon in published work.

4.4 Ethical and Gameplay Fairness Notes

This project trains game NPCs, not surveillance or monitoring systems. The ethical considerations therefore centre on gameplay fairness and player experience rather than privacy or bias in the traditional AI ethics sense.

Aim behaviour is coupled to the learned policy and weapon mechanics, not to perfect tracking. The agent's aim is derived from continuous action outputs that are processed through the same weapon mechanics as a human player. The agent can miss, and does miss, particularly during curriculum transitions when the policy is adapting to new task requirements. This is desirable from a gameplay fairness perspective: an NPC that never misses is frustrating to play against regardless of how it was implemented.

Scripted and learned opponents are kept distinct by curriculum phase. Designers can reason about difficulty at each stage without conflating two different behaviour sources. This separation also enables difficulty tuning: ScriptedBot personality parameters can be adjusted independently of the learned policy.

Manager logic for round outcomes and score tracking is deterministic and not influenced by the learning process. Team assignment, kill counting, round termination, and respawn timing are all governed by explicit code in DeathmatchManager, not by the neural network. This preserves scoring integrity regardless of agent behaviour quality.

The training process itself does not use any player data, personal information, or copyrighted game content. All training occurs in procedurally generated environments against scripted bots or self-play opponents. The resulting ONNX models contain only neural network weights optimised for in-game performance.

4.5 Usability Evaluation

The HUD systems were evaluated through manual gameplay testing across all three modes. The evaluation focused on information readability, update responsiveness, and cognitive load during fast-paced gameplay.

The combat HUD provides immediate feedback on health and ammo state. The colour gradient on the health bar (green to yellow to red) is readable without taking eyes off the centre of the screen, as the colour change is visible in peripheral vision. The ammo counter updates instantly on fire and reload, with the reload progress bar providing clear feedback on reload timing. The hit marker flash (0.15 seconds) is brief enough to not obstruct aiming but visible enough to confirm hits.

The deathmatch HUD scoreboard correctly reflects all 10 team members and updates in real time. Kill events appear in the feed within one frame of the kill event firing. The kill feed text size and team colouring are readable at standard gaming distances.

The racing HUD speed display is readable at both low and high speeds. The numeric display updates at the physics frame rate (50Hz), providing smooth speed indication. Lap and checkpoint counters update without visible delay. The drift indicator activates cleanly when lateral velocity exceeds the threshold and deactivates when the car regains grip.

The TrainingGUI was evaluated by running multiple training campaigns through the interface. The configuration dropdown correctly lists all YAML files. The command preview accurately reflects selected options. The log monitor displays output in real time without lag. The TensorBoard launch button correctly opens TensorBoard in the default browser.

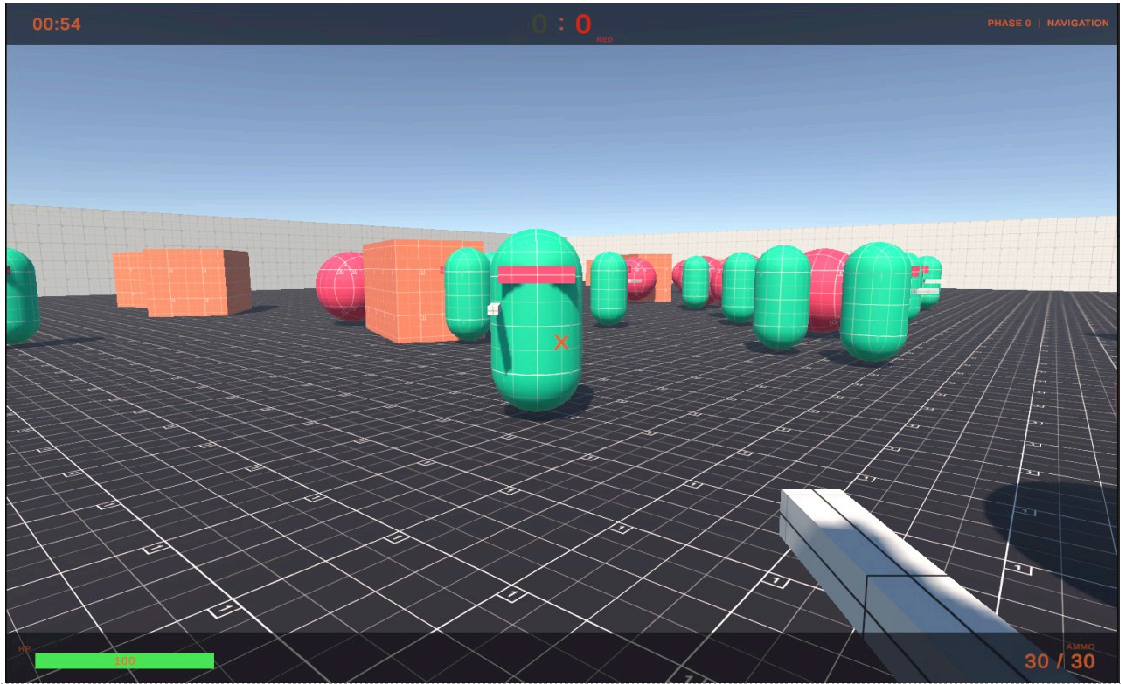


Figure 4.7: Combat HUD showing health bar, ammo count, and hit marker feedback during gameplay

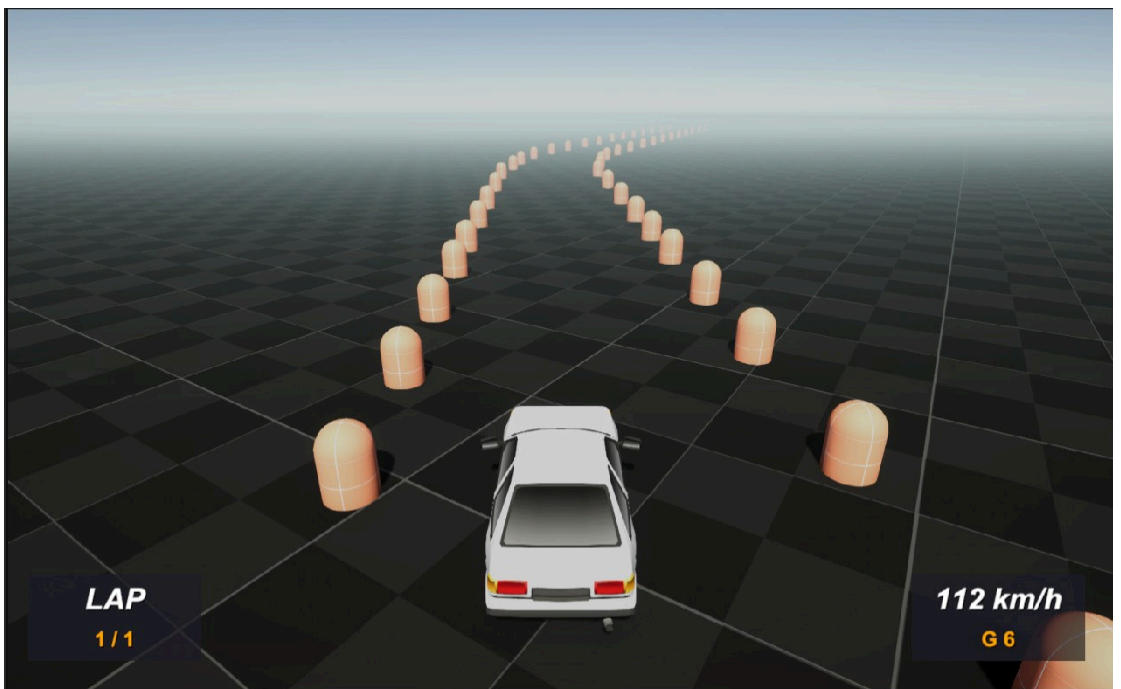


Figure 4.8: Racing HUD displaying speed, lap counter, and position during active race

4.6 Current Limitations

The following limitations were identified during development and validation:

- No archived CarDrivingAgent training_status logs were found in the results directories. The racing evaluation in this report is based on implementation

analysis and manual gameplay validation rather than quantitative checkpoint reward metrics. A dedicated racing training campaign was not completed within the project timeline.

- Some training run histories include resumed checkpoints with null final reward values, indicating that the last checkpoint was saved during a training interruption without a scalar reward update. This affects the `phase1_final` and `phase2_deathmatch` runs.
- Self-play policy quality is sensitive to hyperparameter choices (opponent window size, swap frequency, latest-model ratio). The current settings produce viable behaviour but have not been exhaustively optimised. The ELO oscillation observed in deathmatch runs suggests that additional stability measures (such as league training or exploiter agents) would improve convergence.
- The reward function for combat uses 18 manually tuned parameters. No automated hyperparameter search (grid search, Bayesian optimisation, or population-based training) was performed. The `phase1_v3` reward weights were found through approximately 5 iterations of manual tuning informed by reward curve analysis.
- The procedural arena generation (ArenaBuilder) produces rectangular arenas with box-shaped cover. This limits the environmental diversity available during training. More complex geometry (L-shaped rooms, multi-level structures, destructible cover) would likely produce more robust policies but requires additional development.
- The `CombatAgent`'s observation space does not include teammate state information. In team deathmatch (lesson 5+), the agent cannot directly observe teammate positions, health, or actions. Team coordination emerges only implicitly through shared reward signals, which limits the sophistication of team strategies.
- The current ONNX models are specific to the observation and action space dimensions defined in the training configuration. Changing the observation space (e.g., adding new observation dimensions) requires retraining from scratch. No transfer learning or fine-tuning protocol is implemented.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project set out to design, implement, and evaluate intelligent non-player characters trained through reinforcement learning for two gameplay domains, tactical combat and autonomous racing, within a single Unity 6 project. The work produced a functioning multi-domain NPC framework that combines learned policies with scripted baselines under curriculum control, integrated with production-quality game systems including round management, team assignment, respawn logic, score tracking, and real-time HUD feedback.

The combat intelligence pipeline trains a `CombatAgent` through Proximal Policy Optimisation with LSTM memory and curiosity-driven exploration, progressing through a seven-stage curriculum from basic navigation (lesson 0) to full 5v5 team deathmatch with competitive self-play (lesson 6). The `CombatAgent` observes 37 scalar values supplemented by ray perception data, produces 4 continuous movement and aiming actions alongside 2 discrete combat actions, and receives reward feedback from an 18-parameter reward function that shapes behaviour across navigation, aiming, shooting, positioning, survival, and team coordination.

The racing intelligence pipeline trains a `CarDrivingAgent` with a 25-dimensional observation space combining car state variables, a 17-ray perception fan for track boundary detection, and checkpoint direction vectors. The agent learns throttle, steering, and drift control through a four-stage curriculum progressing from straight-line driving to competitive racing. The `ArcadeCarController` provides deterministic physics that facilitate stable policy convergence.

Among the completed training runs, `phase1_v3` produced the strongest reward profile. The agent reached a final checkpoint reward of approximately 89.96 at step 5,042,737 after 34.4 minutes of training. The curriculum successfully advanced through all five lesson stages (lessons 0 through 5), confirming that the reward shaping and curriculum thresholds are sufficient to drive progressive skill acquisition in the combat environment. The mean episode length of 487.24 steps and policy entropy of 2.64

indicate that the agent maintained diverse exploratory behaviour throughout training rather than collapsing to a narrow deterministic policy.

The self-play deathmatch runs (deathmatch_v3 and phase2_deathmatch) produced the expected non-monotonic reward trajectories and ELO oscillations that are characteristic of competitive multi-agent training. The ELO trajectory reaching 1035.29 at step 4,338,510 indicates that the latest policy consistently outperforms the opponent pool of past checkpoints. The oscillation pattern suggests that the agent cycles through strategies as it co-evolves with its opponents, which is a well-documented phenomenon in self-play literature. Additional training stability measures such as league-based training or exploiter agents would likely reduce this cycling.

The combat_phase1 run (15 million steps, final reward -8.39) demonstrated the consequences of inadequate reward shaping. The death penalty and existential penalty in this early configuration outweighed the kill reward in most combat episodes, causing the agent to learn passive avoidance behaviour rather than active combat engagement. This run directly informed the reward rebalancing in phase1_v3, where positioning rewards were increased and the existential penalty was reduced. The failure case illustrates the literature review finding that reward design dominates convergence quality.

The integration of trained policies into the gameplay loop works correctly across all validated subsystems. The 14 functional test cases documented in Table 22 all passed, covering agent respawn, team assignment, hit detection, damage propagation, kill tracking, kill feed display, scoreboard updates, round termination, checkpoint crossing, lap completion, and all HUD elements. The centralised manager pattern (GameManager, DeathmatchManager, RaceManager) with explicit reset APIs successfully handles episode boundary transitions without disrupting the RL training loop.

The TrainingGUI provides a practical operator workflow for managing training campaigns. Configuration selection, run-id management, command preview, and real-time log monitoring are all accessible through a graphical interface that does not require command-line expertise. The eight custom Unity Editor scripts support development workflows including arena preview, car physics visualisation, HUD element binding, and automated tag setup.

The codebase comprises 43 C# scripts across nine module directories, totalling approximately 8,743 lines of source code. The modular architecture, with clear separation

between the Agent Layer, Environment Layer, Combat Systems Layer, Manager Orchestration Layer, and UI Layer, supports independent development and testing of each subsystem. Four YAML training configurations and six completed training runs provide a documented evidence base for the project outcomes.

In summary, the project demonstrates that curriculum-based reinforcement learning with hybrid scripted-learned opponent scheduling can produce viable combat and racing NPC behaviour within practical training budgets (34 minutes for the strongest combat run), while maintaining clean integration with production-quality game systems. The seven-stage combat curriculum and four-stage racing curriculum represent one of the more extensive curriculum designs in published ML-Agents work. The 5v5 self-play deathmatch with ELO tracking extends the typical scope of Unity ML-Agents projects into team-based competitive training.

5.2 Future Work

The following twelve directions for future development have been identified based on the current limitations, training observations, and literature review. Each direction addresses a specific gap in the current implementation and is ordered by estimated impact on system quality.

5.2.1 Racing Metrics Completion

The most immediate gap is the absence of archived training metrics for the CarDrivingAgent. A dedicated racing training campaign should be run using the `car_racing.yaml` configuration with a standardised run-id scheme, archiving `training_status.json` and `timers.json` artifacts. This would enable the same quantitative analysis (reward trajectories, episode lengths, curriculum progression rates) that was performed for the combat runs, providing a complete evidence base for the racing domain.

5.2.2 Multi-Objective Reward Auto-Tuning

The current 18-parameter combat reward function was tuned through approximately five manual iterations. Automated hyperparameter and reward coefficient sweeps would reduce this tuning burden. Grid search over the most sensitive parameters (death penalty, hit reward, existential penalty) with a coarse grid of 3-5 values per

parameter is the simplest approach. Bayesian optimisation using a framework like Optuna or Ray Tune would be more sample-efficient for the 18-dimensional space. Population-based training (PBT), as used in AlphaStar, would combine hyperparameter search with training itself, adapting reward weights during training rather than before it.

5.2.3 Self-Play Population Management

The current self-play configuration uses a simple opponent window of 10 past checkpoints with a 0.7 latest-model ratio. Implementing league-based training, where the agent trains against a diverse population of past selves plus specialised exploiter agents, would reduce the policy cycling observed in the ELO trajectory. The AlphaStar approach of maintaining main agents, league exploiters, and main exploiters provides a template for this improvement. The key implementation requirement is a checkpoint registry with metadata (ELO at save time, training step, config hash) to enable intelligent opponent selection.

5.2.4 Multi-Scene Domain Randomisation

The current ArenaBuilder produces rectangular arenas with box-shaped cover. Expanding the procedural generation to include L-shaped rooms, multi-level structures, ramps, windows, doorways, and destructible cover would increase the environmental diversity during training. Domain randomisation has been shown to improve policy generalisation to unseen environments. The TrackBuilder could similarly be expanded with more varied track geometries including hairpin turns, banking, split paths, and surface transitions.

5.2.5 Inference Mode Benchmarking

The runtime performance measurements in Section 4.3.2 were taken on development hardware with the Unity Editor running. Standalone build benchmarking on target hardware (consumer-grade GPU and CPU configurations) would provide more realistic performance expectations. Key metrics to measure include ONNX inference latency per agent, frame rate with varying agent counts (1, 5, 10, 20), memory consumption per ONNX model, and the performance impact of enabling or disabling visual effects during inference.

5.2.6 Behaviour Cloning Bootstrap

Imitation learning (behaviour cloning) warm-starts from heuristic or scripted demonstrations could reduce the sparse reward problem in early curriculum stages. ML-Agents supports GAIL (Generative Adversarial Imitation Learning) and BC (Behavioural Cloning) as additional training modes. Recording ScriptedBot behaviour as demonstration data and using it to pre-train the CombatAgent's policy before switching to PPO training could accelerate early curriculum progression and reduce the number of curriculum stages needed.

5.2.7 Tactical Team Communication

The current CombatAgent cannot directly observe teammate state. Adding shared latent vectors (team context) or explicit teammate intent features would enable coordinated squad behaviour. The simplest approach is to add teammate observation dimensions (positions, health, current action) to the observation space. A more sophisticated approach would use a communication channel where agents share learned embeddings, enabling implicit coordination without explicit message design.

5.2.8 Dynamic Difficulty Adaptation

Using policy confidence scores (action probability entropy) and recent K/D metrics to adjust bot aggressiveness at runtime would maintain engagement quality for human players of varying skill levels. When a player is winning easily (high K/D), the system would increase the ScriptedBot personality parameters (higher aim accuracy, shorter reaction time, more aggressive strafe/dodge). When a player is struggling, parameters would relax. The adjustment could also blend between ScriptedBot and trained ONNX agents based on desired difficulty level.

5.2.9 Explainability Tools

Building training dashboards that show per-reward-component contribution over time would help designers reason about which reward terms are driving behaviour changes. The current training output shows only aggregate reward, making it difficult to diagnose why an agent learned a particular behaviour. A reward decomposition tool would track each of the 18 reward terms independently and display them as stacked area

charts, showing the relative contribution of navigation, aiming, hits, kills, damage taken, and other terms across training.

5.2.10 Automated Regression Testing

Adding a repeatable scene-based regression suite would validate that code changes do not break integration boundaries. The suite would use Unity's Test Framework (already included as a package dependency) to run headless tests that verify: agent respawn at correct positions, damage propagation through the full CombatEntity-HitscanWeapon-DamageEventBus pipeline, curriculum lesson switching with correct entity activation, round termination at kill limit and timer expiry, and checkpoint progression in the racing domain.

5.2.11 Model Versioning

Building a model registry with metadata (config hash, reward profile at save time, scene compatibility, training step count, ELO rating) would enable safer deployment and rollback of trained policies. The current approach stores ONNX files in per-run directories without formal versioning. A registry would support A/B testing between model versions, automated selection of the best-performing checkpoint per configuration, and compatibility checking between model input dimensions and scene observation space.

5.2.12 Production Packaging

Creating a deployment package including selected ONNX models, configuration presets, difficulty profiles, and in-game model selection controls would make the system distributable to end users. The package would include a settings menu allowing players to select NPC difficulty (ScriptedBot easy/medium/hard, trained ONNX model), adjust team sizes, configure round parameters, and switch between combat and racing modes. This packaging step represents the bridge from research prototype to shippable game feature.

REFERENCES

- [1] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G. and Petersen, S. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), pp.529-533. doi:10.1038/nature14236.
- [2] Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M. and Dieleman, S. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), pp.484-489. doi:10.1038/nature16961.
- [3] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A. and Chen, Y. (2017). Mastering the game of Go without human knowledge. *Nature*, 550(7676), pp.354-359. doi:10.1038/nature24270.
- [4] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347.
- [5] Juliani, A., Berges, V.P., Teng, E., Cohen, A., Harper, J., Elion, C., Goy, C., Gao, Y., Henry, H., Mattar, M. and Lange, D. (2018). Unity: A General Platform for Intelligent Agents. arXiv preprint arXiv:1809.02627.
- [6] Vinyals, O., Babuschkin, I., Czarnecki, W.M., Mathieu, M., Dudzik, A., Chung, J., Choi, D.H., Powell, R., Ewalds, T., Georgiev, P. and Oh, J. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782), pp.350-354. doi:10.1038/s41586-019-1724-z.
- [7] Berner, C., Brockman, G., Chan, B., Cheung, V., Debiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C. and Józefowicz, R. (2019). Dota 2 with Large Scale Deep Reinforcement Learning. arXiv preprint arXiv:1912.06680.
- [8] Baker, B., Kanitscheider, I., Marber, T., Vitchyr, H., Mordatch, I. and Evans, R. (2020). Emergent Tool Use From Multi-Agent Autocurricula. In *Proceedings of the International Conference on Learning Representations (ICLR 2020)*.
- [9] Sutton, R.S. and Barto, A.G. (2018). *Reinforcement Learning: An Introduction*. 2nd edition. Cambridge, MA: MIT Press. ISBN: 978-0262039246.

- [10] Pathak, D., Agrawal, P., Efros, A.A. and Darrell, T. (2017). Curiosity-driven Exploration by Self-Supervised Prediction. In Proceedings of the 34th International Conference on Machine Learning (ICML 2017), pp.2778-2787.
- [11] Bengio, Y., Louradour, J., Collobert, R. and Weston, J. (2009). Curriculum Learning. In Proceedings of the 26th International Conference on Machine Learning (ICML 2009), pp.41-48. doi:10.1145/1553374.1553380.
- [12] Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), pp.1735-1780. doi:10.1162/neco.1997.9.8.1735.
- [13] Bansal, T., Pachocki, J., Sidor, S., Sutskever, I. and Mordatch, I. (2018). Emergent Complexity via Multi-Agent Competition. In Proceedings of the International Conference on Learning Representations (ICLR 2018).
- [14] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D. and Riedmiller, M. (2013). Playing Atari with Deep Reinforcement Learning. arXiv preprint arXiv:1312.5602.
- [15] Haarnoja, T., Zhou, A., Abbeel, P. and Levine, S. (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In Proceedings of the 35th International Conference on Machine Learning (ICML 2018), pp.1861-1870.
- [16] Matarić, M.J. (1994). Reward Functions for Accelerated Learning. In Proceedings of the 11th International Conference on Machine Learning (ICML 1994), pp.181-189.
- [17] Isla, D. (2005). Handling Complexity in the Halo 2 AI. In Proceedings of the Game Developers Conference (GDC 2005).
- [18] Mark, D. (2009). Behavioral Mathematics for Game AI. Boston, MA: Charles River Media. ISBN: 978-1584506843.
- [19] Unity Technologies. (2024). Unity ML-Agents Toolkit Documentation. Available at: <https://github.com/Unity-Technologies/ml-agents> [Accessed: March 2026].
- [20] Unity Technologies. (2024). Unity Manual and Scripting API Reference. Available at: <https://docs.unity3d.com> [Accessed: March 2026].
- [21] OpenAI. (2018). OpenAI Five: Dota 2 AI. Available at: <https://openai.com/research/openai-five> [Accessed: February 2026].

- [22] Schulman, J., Levine, S., Abbeel, P., Jordan, M.I. and Moritz, P. (2015). Trust Region Policy Optimization. In Proceedings of the 32nd International Conference on Machine Learning (ICML 2015), pp.1889-1897.
- [23] Van Hasselt, H., Guez, A. and Silver, D. (2016). Deep Reinforcement Learning with Double Q-Learning. In Proceedings of the 30th AAAI Conference on Artificial Intelligence, pp.2094-2100.
- [24] Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M. and Freitas, N. (2016). Dueling Network Architectures for Deep Reinforcement Learning. In Proceedings of the 33rd International Conference on Machine Learning (ICML 2016), pp.1995-2003.
- [25] Schaul, T., Quan, J., Antonoglou, I. and Silver, D. (2016). Prioritized Experience Replay. In Proceedings of the International Conference on Learning Representations (ICLR 2016).
- [26] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T. and Lillicrap, T. (2018). A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go Through Self-Play. Science, 362(6419), pp.1140-1144.
- [27] Jaderberg, M., Czarnecki, W.M., Dunning, I., Marris, L., Lever, G., Castaneda, A.G., Beattie, C., Rabinowitz, N.C., Morcos, A.S., Ruderman, A. and Sonnerat, N. (2019). Human-level Performance in 3D Multiplayer Games with Population-Based Reinforcement Learning. Science, 364(6443), pp.859-865.
- [28] Ho, J. and Ermon, S. (2016). Generative Adversarial Imitation Learning. In Advances in Neural Information Processing Systems (NeurIPS 2016), pp.4565-4573.
- [29] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W. and Abbeel, P. (2017). Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. In IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2017), pp.23-30.
- [30] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. and Desmaison, A. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In Advances in Neural Information Processing Systems (NeurIPS 2019), pp.8026-8037.

coursera

Mar 13, 2026

Manish

has successfully completed

AI Agents: Multi-Agent Design & Governance

an online course authorized by Coursera and offered through Coursera

Gleb Marchenko

5G/6G technical trainer and advisor

Starweaver Instructor Team



COURSE
CERTIFICATE



Verify at:
<https://coursera.org/verify/7M2P98JLXK91>

Coursera has confirmed the identity of this individual and their participation in the course.

This certificate attests to the learner's completion of an online course / project delivered via Coursera. It does not constitute formal enrollment at any university or entity and does not itself grant academic credit, grades, or a degree. Institutions or organizations may, at their discretion, recognize this learning toward their own programs or credentials.